

VCF-RDFizer: A Conversion Tool to Represent Variant Data as a Knowledge Graph

Elias Crum^{ORCID^{1,2*}}, Bart Buelens^{ORCID¹}, Ruben Taelman^{ORCID²}, Gökhan Ertaylan^{ORCID¹}

²IDLab, Department of Electronics and Information Systems, Ghent University – imec, Technologiepark-Zwijnaarde 116, 9052, Belgium.

¹Digital Biological Systems, Flemish institute for Technological Research (VITO), Boeretang 200, 2400, Belgium.

*Corresponding author(s). E-mail(s): elias.crum@ugent.be;

Abstract

Purpose: The Variant Call Format (VCF) is the dominant standard for representing genomic variation and underpins population genomics, clinical sequencing, and precision-medicine workflows. As VCF datasets are increasingly interpreted together with sample metadata, clinical observations, public biomedical resources, and governance information, a Resource Description Framework (RDF) representation can provide a common graph model for linkage and declarative querying.

Methods: We present VCF-RDFizer, a command-line tool distributed through PyPI, Conda-Forge, and Docker that converts VCF into RDF using a public VCF-oriented vocabulary and modular RML mappings. In addition to raw N-Triples and general-purpose gzip/Brotli artifacts, the current workflow produces two queryable compressed representations: HDT and COTTAS. A storage-aware chunk pipeline incrementally reads a plain or concatenated-gzip RDF aggregate, materializes only one newline-safe uncompressed chunk at a time, converts that chunk into the selected representation(s), deletes the chunk, and merges the representation parts. We further define six paired bioinformatic queries to be evaluated over RDF with SPARQL and over the source VCF with VCF-native software to test representation equivalence.

Results: The completed baseline benchmark reported in this draft covers six original VCF inputs and demonstrates tractable RDF conversion on workstation-class hardware. The reproducibility corpus has now been expanded to ten inputs, adding a phased 2,504-sample 1000 Genomes chromosome-20 VCF, a 32-sample HGSVC2 structural-variant VCF, and two Genome in a Bottle v4.2.1 benchmark

VCFs. Current-pipeline HDT/COTTAS measurements and SPARQL-versus-VCF-parser query measurements are not yet reported; placeholders are retained to prevent unsupported performance or equivalence claims.

Conclusion: VCF-RDFizer provides a reproducible route from VCF to interoperable and optionally queryable compressed RDF, supporting both semantic integration and testable preservation of core variant and genotype information.

Keywords: Knowledge Representation, Genomic Variant Data, Variant Call Format, RDF, SPARQL, HDT, COTTAS, Semantic Validation

Revision-marking key. Blue text denotes additions or substantively revised passages. Red strikethrough, where used, denotes superseded wording. Orange [AUTHOR QUERY] statements identify information that is not supported by the supplied artifacts and must be resolved before submission.

[AUTHOR QUERY: Please confirm whether all ten inputs have completed conversion with the current dev implementation. If so, replace the cautious Results wording in the abstract with the measured ten-file findings and add the corresponding numerical ranges.]

1 Introduction

Variant Call Format. The Variant Call Format (VCF) is the dominant standard for representing and exchanging genomic variants across research and clinical sequencing pipelines, including pharmacogenomics and precision medicine applications [1–3]. As clinical sequencing expands, variant datasets are increasingly interpreted alongside external biomedical resources, patient or sample metadata, and governance constraints controlling access to sensitive genomic information. Operational semantic-genomics initiatives already demonstrate the value of linking variant observations to curated biomedical databases and controlled-access frameworks [4, 5].

These emerging use cases expose limitations of the traditional VCF representation. First, clinical interpretation frequently requires linking variants to external knowledge sources such as protein annotations, disease databases, or pharmacogenomic resources. While VCF supports annotation fields, it provides limited mechanisms for structured, machine-interpretable linkage between variant records and external resources. Second, genomic datasets frequently require fine-grained privacy controls, such as restricting access to particular genomic regions or subsets of variants, which the VCF specification does not natively support. Third, querying variant collections across multiple VCF files, for example, identifying clinically relevant mutations or retrieving variants within genomic position intervals, typically relies on specialized command-line utilities (e.g., bcftools [6]) or custom pipelines rather than interoperable, declarative query mechanisms.

In other domains, semantic web and policy frameworks, such as Linked Data [7] and the Open Digital Rights Language (ODRL) [8], enable structured linkage between datasets, external knowledge resources, and access control policies. Applying similar

approaches to genomic variant data could enable richer integration with biomedical knowledge bases and more expressive query capabilities.

Resource Description Framework and Knowledge Graphs. The Resource Description Framework (RDF) [9] provides a widely adopted graph-based data model for representing relationships between entities identified by globally unique identifiers (IRIs). Such graph-structured data can be organized as *knowledge graphs*, in which nodes represent entities of interest and edges capture relationships between them, with the graph serving to accumulate and convey knowledge [10]. For variant data, RDF enables (i) structured linkage between variant data and external biomedical resources [11–13], (ii) native declarative querying via the SPARQL query language [14], and (iii) fine-grained privacy policy enforcement over variant data and linkages [8]. In practice, this could enable, for instance, a user to query a collection of VCF-derived datasets to identify variants whose alternative alleles occur in a specific gene, are associated with a given phenotype, and are accessible under particular policy restrictions.

From VCF to RDF. Transforming VCF into RDF requires *semantic lifting*: converting the row-based structure of VCF files into graph statements. Relevant VCF elements such as genomic position, alleles, genotypes, and sample identifiers are assigned IRIs, and fields are translated into subject–predicate–object triples. Ontology-aligned predicates provide consistent semantics which enable datasets to be queried and linked.

This transformation requires two components: (i) a semantic vocabulary describing VCF concepts and relationships, and (ii) a mapping process that converts VCF file content into RDF using this vocabulary. Such mappings must be systematic and reproducible due to the structural heterogeneity permitted by the VCF specification [1]. INFO fields may contain semi-structured annotations, FORMAT and genotype columns vary across datasets, and multi-sample files aggregate multiple genotypes within shared variant records, complicating the creation of consistent semantic representations.

Several efforts have explored semantic representations of VCF and genomic-variation data, ranging from direct VCF-to-RDF converters to systems that expose genomic variation through RDF and SPARQL interfaces.

Early approaches include the *rdf-vcf* reader and *VCFr*, the latter introducing an isomorphic mapping between VCF and RDF/JSON-LD [16, 19]. Dedicated conversion utilities include JVarKit’s *VcfToRdf*, the Rust-based *VCF2RDF* utility developed for TOGOvar [4, 15], and the *vcf2rdf* component of SPARQLing Genomics [17, 21]. The latter has also been used in more recent genomic knowledge-graph applications, including VariantKG [22]. More general VCF-processing frameworks such as *bio-vcf* can additionally serialize VCF-derived information as RDF or JSON-LD through configurable output templates [18]. In contrast to these materializing approaches, *Semantic Beacons* uses declarative RML mappings to expose genomic-variation data obtained through the Beacon API as RDF, enabling federated SPARQL access without functioning as a conventional VCF-to-RDF converter [20].

Table 1 compares these approaches according to their semantic modeling, transformation workflow, support for metadata and genotype information, extensibility,

Table 1 Comparison of VCF-to-RDF tools and related semantic genomic-variation workflows against the capabilities considered in this study. Approach distinguishes dedicated VCF-to-RDF converters from general VCF-processing tools and systems that expose genomic variation through virtual RDF access. Columns indicate whether a system provides: a command-line interface (CLI) for direct pipeline use; a publicly documented RDF model or vocabulary describing the generated representation (Documented RDF model); a documented and reproducible workflow from genomic-variation input to RDF access or materialization (Reprod. workflow); support for representing VCF metadata together with genotype and sample-level information (Metadata + genotype); user-extensible transformation rules or mappings that can be modified without changing the core converter implementation (Extensible mappings); integrated support for reducing RDF storage requirements through compressed RDF representations or compression-aware processing (RDF compression); and a publicly distributed reusable software artifact such as a package, executable, container, or maintained application distribution (Public distribution). Status codes: Y = explicitly supported and documented; P = partially supported, configurable, indirect, or limited to a subset of the relevant functionality; N = no integrated support identified in the publicly available documentation reviewed.

Tool/workflow	Approach			Documented	Reprod.	Metadata +	Extensible	RDF	Public
				CLI	workflow	genotype	mappings	compression	distribution
VCF-RDFizer (this study)	(this study)	Dedicated converter	con-	Y	Y	Y	Y	Y	Y
VCF2RDF [4]	(TOGOvar)	Dedicated converter	con-	Y	P	Y	P	N	Y
JVarkit VcfToRdf [15]		Dedicated converter	con-	Y	P	Y	P	N	Y
VCFr / VCF2RDF [16]		Web-based converter	con-	P	Y	P	P	N	P
SPARQLing vcf2rdf [17]	Genomics	Dedicated converter	con-	Y	P	Y	Y	P	P
bio-vcf [18]		General processor	VCF	Y	N	Y	P	Y	N
ruby-rdf/rdf-vcf [19]		Legacy reader/converter	RDF	Y	N	P	P	N	Y
Semantic Beacons [20]		Virtual semantic access		N	P	P	P	Y	N

compression, and software distribution. The comparison illustrates that previous systems address different portions of the VCF-to-semantic-data workflow. Some prioritize lightweight and efficient conversion, others provide highly customizable serialization or virtual semantic access, while several expose only a subset of VCF structure or rely on implementation-specific RDF models. VCF-RDFizer is designed to combine these concerns within a single reproducible workflow by providing a publicly documented VCF semantic model, declarative and extensible mappings, systematic representation of record-, metadata-, and sample-level information, and integrated production of both conventional and compressed queryable RDF representations.

Open Challenges.

First, existing VCF-to-RDF approaches differ substantially in how explicitly and completely they define the semantics of the generated RDF. Some systems document an RDF representation or mapping of VCF concepts, such as the isomorphic VCF-to-RDF model proposed by VCFr [16], while others use lightweight or implementation-specific RDF structures optimized for particular conversion or application requirements [4, 15]. More general tools such as *bio-vcf* allow users to define custom RDF serializations, but do not prescribe a reusable semantic model of VCF structure.

Consequently, a remaining challenge is the availability of a standalone, publicly documented vocabulary that systematically captures the structural semantics of VCF itself, including file- and header-level metadata, variant records, INFO and FORMAT definitions and values, sample-level calls, and genotype relationships. Existing genomic ontologies and data models such as GA4GH Variant Representation [23], FALDO [24], the Sequence Ontology [25], GENO, and SNPO provide important semantics for genomic locations, variation, and biological entities, but were not designed to represent the complete structure and serialization semantics of a VCF document. A dedicated VCF semantic layer can therefore complement these resources while reusing their established biological concepts where appropriate.

Second, extensibility is supported through different mechanisms across existing converters, but semantic transformation rules are often coupled to implementation logic, configuration files, or application-specific templates. For example, TOGOvar’s *VCF2RDF* exposes configurable namespaces, INFO-field selection, and reference mappings through YAML configuration [4], while *bio-vcf* provides highly flexible output templates capable of generating customized RDF representations. Such mechanisms are useful for adapting output, but the resulting transformation semantics remain closely associated with the particular software or template implementation.

Declarative mapping languages provide a complementary approach by representing transformation rules independently from the execution engine. RML (RDF Mapping Language) [26] supports mappings from heterogeneous data sources into RDF and enables mapping definitions to be reused, inspected, versioned, and extended independently of the software executing them. This approach is also employed by Semantic Beacons, which uses RML mappings to transform Beacon API responses into RDF for on-demand semantic access [20]. The RML ecosystem additionally includes engines such as RMLStreamer [27], which support scalable execution of declarative mappings. For VCF conversion, separating the semantic model and mapping rules from procedural parsing logic provides a mechanism for adapting dataset-specific INFO, FORMAT, metadata, or ontology alignments without requiring modifications to the core conversion implementation.

Third, most existing VCF-to-RDF workflows do not integrate strategies for addressing the storage expansion introduced by semantic lifting. Expanding compact genomic formats such as VCF into explicit RDF triples can substantially increase storage requirements, particularly because identifiers and structural relationships that are implicit in VCF become explicit and may be repeated across large numbers of triples [9].

Compression strategies therefore play an important role, but archival and queryable compression address different requirements. General-purpose compressors such as gzip and Brotli [28] can substantially reduce storage and transfer costs, but the resulting artifacts ordinarily require decompression before they can be queried as RDF. In contrast, RDF-specific representations such as HDT (Header–Dictionary–Triples) [29] combine compression with indexed triple-pattern access. More recently, COTTAS represents RDF as a columnar triple table using Apache Parquet and block-range indexes, enabling triple-pattern evaluation directly over the compressed representation [30]. Integrating both archival and queryable representations within a VCF-to-RDF

workflow therefore allows users to select storage strategies according to whether the resulting genomic knowledge graph is intended primarily for distribution, long-term storage, direct querying, or a combination of these purposes.

Study Aims. Taken together, these limitations motivate the development of a reusable and reproducible VCF-to-RDF transformation tool whose output can be independently validated. Here we present *VCF-RDFizer*, which integrates a publicly documented vocabulary, modular RML-based mappings, storage-aware conversion, general-purpose compression, and queryable HDT and COTTAS representations. As summarized in Table 1, VCF-RDFizer combines capabilities that are typically disconnected in existing approaches: a standalone CLI <https://github.com/ecrum19/VCF-RDFizer>, a public vocabulary <https://w3id.org/vcf-rdfizer/vocab#>, modular declarative mappings, a reproducible pipeline, queryable compressed outputs, and public packaging <https://pypi.org/project/vcf-rdfizer/>. In addition to measuring conversion and compression costs, this study defines a paired SPARQL/VCF-parser workload to determine whether the RDF representation reproduces established variant and genotype summaries exactly.

2 VCF-RDFizer CLI Tool

VCF-RDFizer is a command-line tool that converts input VCF files into semantically enriched RDF using a reproducible mapping pipeline. The workflow integrates pre-processing, mapping, RDF conversion, and optional compression/decompression functionalities. An overview of these workflows is shown in Figure 1.

[AUTHOR QUERY: The caption now describes COTTAS and the one-raw-chunk-at-a-time workflow, but the supplied vcf2rdf.pdf artwork was not available for editing. Please regenerate Figure 1 so that the diagram

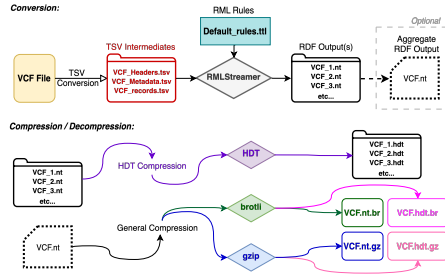


Fig. 1 VCF-RDFizer Pipeline Workflow The flowchart depicts end-to-end conversion from VCF input to semantically enriched, compressed RDF outputs. **Conversion:** VCF-RDFizer converts VCF file into TSV intermediates, uses RML mappings and RMLStreamer to generate N-Triples, and optionally aggregates VCF RDF outputs to a single N-Triples (.nt) file. **Compression and representations:** VCF-RDFizer can emit general-purpose gzip/Brotli artifacts and queryable HDT and COTTAS representations. HDT and COTTAS artifacts may additionally be packaged with gzip or Brotli for distribution; those packaged artifacts must be unpacked before direct querying. **Storage-aware construction:** In space-optimized mode, a concatenated-gzip N-Triples aggregate is read incrementally, only one uncompressed record-safe chunk is materialized at a time, each selected representation consumes that chunk, and the chunk is deleted before the next is produced. **Decompression and validation:** General-purpose packaging is reversible, and generated HDT/COTTAS artifacts are decoded through their native readers for triple-count validation.

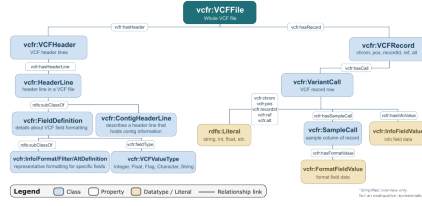


Fig. 2 VCF-RDFizer Vocabulary Semantic Model (simplified) The figure depicts a simplified representation of the semantic hierarchy of core terms of the VCF-RDFizer vocabulary. All *vcfr:* prefixes are an abbreviation for <https://w3id.org/vcf-rdfizer/vocab#>. Genotype/sample class relationships are not represented in this figure for simplicity, but more details are available in the interactive representation of the vocabulary at: <https://ecrum19.github.io/VCF-RDFizer-vocabulary/>.

itself shows the new storage modes, COTTAS branch, balanced merges, validation, and packaging distinction.]

2.1 Semantic Model

The semantic model we developed to represent VCF data combines a dedicated VCF-RDFizer vocabulary (<https://w3id.org/vcf-rdfizer/vocab#>) with terms aligned with established biological ontologies to maximize interoperability and reuse. Term equivalencies are defined for genomic coordinates (FALDO [24]), variant typing (the Sequence Ontology [25]), genotype/sample constructs (HERO [31]), and features of the GA4GH VRS schema v2.0.1 [32]. This hybrid strategy retains VCF-specific semantics while avoiding duplication of mature ontology terms.

The model preserves three information layers: metadata/header assertions, variant-level assertions, and genotype/sample assertions. Core entities are minted using deterministic IRI templates derived from stable VCF attributes (e.g., chromosome, position, reference allele, etc.), which keeps identifiers reproducible across runs. The vocabulary website also provides a suggested canonical IRI pattern for instances of the core classes, which can be used as a reference for users who want to generate their own RDF from VCF files using the provided vocabulary and mappings. A visual representation of the core components of the vocabulary is shown in Figure 2, and a more detailed and interactive version is available on the companion website: <https://ecrum19.github.io/VCF-RDFizer-vocabulary/>. On the website are also examples of how a VCF file will look after conversion to RDF (as well as an example for multi-sample cases).

2.2 Tool Workflow

VCF-RDFizer is implemented as a Python command-line tool that coordinates five stages: (1) conversion of each VCF into records, header-line, and file-metadata TSV logical sources; (2) TSV-to-RDF transformation with modular RML mappings and RMLStreamer; (3) construction of one logical N-Triples aggregate in either plain or space-optimized storage mode; (4) optional creation of raw-RDF gzip/Brotli artifacts and/or queryable HDT and COTTAS representations; and (5) native representation

validation, packaging, cleanup, and run-metric reporting. The command-line interface separates these concerns through `--rdf-storage-mode`, `--rdf-compression`, `--representations`, and `--artifact-compression`. Thus, an unwrapped `.hdt` or `.cottas` file can be retained for querying while an additional `.gz` or `.br` package is created for transport or archival storage.

Mapping rules are authored in Turtle as modular RML rule sets that mediate the conversion from VCF to RDF. This modularity allows rules to be extended or modified without rewriting the full pipeline. To execute the mappings, we use RMLStreamer [27]; the baseline experiments used version 2.5.0 through its standalone command-line interface.

Aggregate storage modes. In plain mode, RMLStreamer parts are appended to one uncompressed `<sample>.nt` aggregate and deleted after consumption. In `space-optimized` mode, each line-oriented RMLStreamer part is gzip-compressed, appended as a member of one concatenated-gzip `<sample>.nt.gz` stream, and then deleted. Standard gzip readers expose the concatenated members as one logical N-Triples sequence. This avoids retaining all RMLStreamer parts and a second full uncompressed aggregate simultaneously.

One-chunk-at-a-time representation construction. The current implementation incrementally reads the plain or gzip aggregate, respects newline-terminated N-Triples record boundaries, and materializes one raw chunk in a Docker-managed workspace. Each selected representation builder consumes that chunk before the raw chunk is deleted and the next chunk is created:

`.nt.gz aggregate → one raw chunk → HDT/COTTAS conversion → delete chunk → ...`

Consequently, at most one uncompressed RDF chunk exists at a time. The final HDT/COTTAS parts, merge intermediates, indexes, and converter scratch space still require storage, but the pipeline no longer creates a full decompressed duplicate of the aggregate.

HDT construction. Each chunk is converted with `rdf2hdt`. Intermediate HDT files are merged pairwise in a balanced tree with `HDTCat` [33]; an unpaired file is carried into the next merge round. After the final HDT is produced, it is opened with the bundled HDT Java reader to create the final sibling `.hdt.index.v1-1` index without executing a large query. Intermediate indexes are not retained because the final merged dictionary and triple layout are authoritative.

COTTAS construction. COTTAS encodes RDF in an Apache Parquet triple table and supports compressed triple-pattern evaluation through columnar pruning and indexes [30, 34]. VCF-RDFizer invokes `pycottas.rdf2cottas` with `disk=True` for each chunk and merges intermediate files pairwise with `pycottas.cat` with `remove_input_files=True`. The final merged COTTAS output rebuilds its selected `spo` indexes. Each `pycottas` operation runs in an isolated temporary working directory so that its local DuckDB scratch database cannot be accidentally reused by a later chunk or merge.

Validation and metrics. Before packaging or raw-RDF cleanup, every base HDT or COTTAS artifact is opened with its native reader and decoded as a stream. The decoded triple count is compared with the source count recorded by RMLStreamer

or the partition planner; no full decoded RDF file is written for this check. A missing decoder, unreadable artifact, or count mismatch fails the representation stage and prevents source cleanup. Chunk conversion, merging, final indexing, validation, and packaging are timed separately, while sample-level metrics report cumulative wall/user/system time, maximum resident set size, output sizes, and cleanup status.

Gzip and Brotli [28] remain available for raw RDF and artifact packaging. The baseline experiments used Brotli level 7 of 11 because higher levels increased runtime with little observed size improvement. For constrained workspaces, the documented lower-peak fallback sets `--chunk-min-bytes 67108864`, `--chunk-target-bytes 134217728`, and `--chunk-max-bytes 268435456` (64, 128, and 256 MiB, respectively), trading additional chunks and merges for lower temporary peak usage.

[AUTHOR QUERY: Please confirm (i) the exact VCF-RDFizer commit or release to cite, (ii) the installed HDT Java/rdf2hdt/HDTCat and pycottas versions, (iii) the default chunk-size values, and (iv) the chunk-size values actually used for the new experiments. The 64/128/256 MiB values above are documented as a low-peak fallback, not assumed benchmark settings. Also confirm that every source-checkout rerun used --build and record the resulting Docker image digest.]

The implementation and default RML mappings are available at <https://github.com/ecrum19/VCF-RDFizer>.

From a user-experience perspective, each tagged VCF-RDFizer version is published across PyPI, Conda-Forge, and DockerHub so users can choose pip-, conda-, or container-based installation with matched versions. The GitHub README serves as the primary user guide, including quick-start commands, end-to-end usage examples, command-line option documentation, and troubleshooting notes. For maintainability, the project follows a release-oriented workflow with versioned changelogs, public issue triage, planned periodic dependency/container updates, and extensive unit-testing with the tool to preserve reproducibility over time.

2.3 Tool Testing

VCF Test Data. The original performance baseline used five publicly accessible Personal Genome Project-associated VCFs [35] and one HG002 precisionFDA/Genome in a Bottle challenge VCF [36]. The reproducibility corpus has now been expanded with four direct-download public files chosen to test structures not represented adequately by the original baseline: (i) the phased 1000 Genomes Phase 3 chromosome-20 call set with 2,504 samples on GRCh37 [37]; (ii) an HGSVC2 structural-variant call set with 32 samples on GRCh38, using sequence-resolved REF/ALT alleles [38]; and (iii-iv) the HG004 and HG005 Genome in a Bottle v4.2.1 single-sample GRCh38 benchmark VCFs covering chromosomes 1-22 [39]. Table 2 distinguishes the completed six-file baseline from the expanded validation and stress-test inputs. Download commands and filename normalization are versioned in the testing repository: <https://github.com/ecrum19/vcf-rdfizer-testing>.

The size discrepancy is resolved: the column labelled `.vcf.gz` in Table 4 reports the *uncompressed* stream, which is the quantity the tool records as `input_vcf_size_bytes`. Two independent checks confirm this. The retained

Table 2 VCF-RDFizer ten-file reproducibility corpus. Approximate compressed sizes are those documented by the testing repository and are not substituted into the legacy benchmark tables. “Header preflight” indicates metadata that must be read from the source VCF rather than inferred from a provider profile.

Canonical input	Approx. MB	Assembly	Samples	Evaluation role
NG1N86S6FC.vcf.gz	379	Header preflight	TBD*	Original personal-WGS conversion baseline; heterogeneous meta-data/annotation profile
NG131FQA1I.vcf.gz	224	Header preflight	TBD*	Original personal-WGS conversion baseline
NB72462M.vcf.gz	341	Header preflight	TBD*	Original personal-WGS conversion baseline
60820188475559.vcf.gz	325	Header preflight	TBD*	Original filtered-SNP profile
60820188474283.vcf.gz	222	Header preflight	TBD*	Original personal-WGS conversion baseline
OG00R_HG002.vcf.gz	69	Header preflight	1	Original HG002 precisionFDA/GIAB challenge baseline
1000G_phase3_chr20.vcf.gz	327	GRCh37	2,504	Phased population-scale multi-sample stress case; genotype aggregation and call-rate queries
HGSVC2_freeze3_sv_alt.vcf.gz	31.5	GRCh38	32	Multi-sample structural-variant and long sequence-allele stress case
HG004_GRCh38_1.22.v4.2.1.benchmark1.vcf.gz	149	GRCh38	1	Independent GIAB benchmark genome across autosomes
HG005_GRCh38_1.22.v4.2.1.benchmark39.vcf.gz	139	GRCh38	1	Independent GIAB benchmark genome across autosomes

*The existing draft described the first five provider/profile inputs as single-sample. The attached testing specification recommends verifying sample count, assembly, GT availability, and header conventions from each VCF header rather than assuming them from provenance pages.

NG131FQA1I.vcf is 1,215,509,730 bytes uncompressed, exactly the 1215.5 MB the legacy table reports for that input; and the current campaign records HG005_GRCh38 as 2,359,594,728 bytes against a 139 MB distributed .vcf.gz, a ratio consistent with VCF gzip. The legacy column heading is therefore mislabelled rather than the values being wrong, and it should read “input size (uncompressed, MB)”. Header preflight of that same retained file gives VCFv4.2, one sample (NG131FQA1I), and GRCh38 contigs. The remaining eight provider files were not retained locally, so their preflight values still require capture from the source downloads.

[AUTHOR QUERY: Please capture assembly, sample count and ##fileformat for the eight provider files that were not retained locally, and correct the column heading of Table 4 to state uncompressed size.]

The first six inputs remain the provenance of the numerical baseline results currently shown in Tables 4 and 5. The four added inputs broaden the test design but

must not be described as completed benchmark runs until their conversion, compression, validation, and query artifacts are present in the archived experiment manifests.

[AUTHOR QUERY: Which of the four added inputs have completed (a) RDF conversion, (b) HDT construction, (c) COTTAS construction, and (d) all six paired queries? Please provide the finished experiment/run names so the Results placeholders can be populated without guessing.]

VCF Version Coverage. Every input in the corpus declares a `##fileformat` of either VCFv4.1 or VCFv4.2, nine at VCFv4.2 and the 1000 Genomes Phase 3 chromosome-20 call set at VCFv4.1. This distribution reflects the state of published variant data rather than a restriction imposed during corpus construction. Although the specification has since advanced through VCFv4.3, VCFv4.4, and VCFv4.5 [1], we were unable to identify any large public dataset distributed above VCFv4.2. A survey of the `##fileformat` declarations of major public variant resources, conducted on 8 September 2026, returned VCFv4.2 for dbSNP build 156 [40] and gnomAD v4.1 [41], and VCFv4.1 for ClinVar [42], the 1000 Genomes 30x GRCh38 phased call set [43], and Ensembl variation [44]. Conformance to VCFv4.3 and above can therefore presently be exercised only against the example and conformance files distributed with the specification itself [1], and not against production-scale data. The corpus consequently characterizes VCF-RDFizer on the versions that data producers actually emit, and support for the newer revisions, in particular the local-allele `FORMAT` fields introduced in VCFv4.4 and VCFv4.5, remains to be evaluated separately.

[AUTHOR QUERY: A stronger draft formulation of the paragraph above asserted that “essentially every caller (DeepVariant, DRAGEN, GATK, Sniffles) still emits VCFv4.2”. That caller-level claim could not be substantiated for this revision: the version literal does not appear in the VCF-related source files of the public DeepVariant or Sniffles repositories, and DRAGEN is closed-source. The wording above is therefore deliberately limited to the reference resources whose headers were measured directly. If you wish to retain the caller-level statement, please supply per-tool evidence (release notes, or `##fileformat` lines from your own outputs), as it would otherwise be an unsupported generalization.]

Testing Environment The hardware and software description below applies to the completed six-file legacy baseline unless a current-pipeline rerun manifest explicitly records the same conditions. VCF-RDFizer automatically logs wall time, user time, CPU time, peak memory (via resident set size (RSS)), output directory size, and total triple counts, along with many other metrics, to support reproducible performance comparisons across datasets and hardware setups. All metrics are collected via bash scripts at tool run-time to ensure reproducibility and consistent measurement across all experiments. Output validation in this study included syntax checks and targeted inspection of variant, genotype, and metadata nodes to verify that semantic links and datatypes were generated as intended.

The legacy experiments were conducted on a workstation running Ubuntu 24.04.4 LTS with an Intel(R) Xeon(R) Silver 4309Y CPU @ 2.80GHz (8 cores, 2.8 GHz), 33.6 GB RAM and a 210 GB SSD, Kernel version 6.8.0-45-generic. Software versions:

Table 3 Paired semantic-equivalence query workload. Each SPARQL result is normalized and compared with an independently calculated VCF-side result for the same source and explicitly matched scope.

ID	Operation	VCF/RDF elements tested	Real-world purpose	VCF-side oracle
Q1	Record density by contig and 1-Mb window	CHROM, POS; vcfr:chrom, vcfr:pos	Locates genomic windows with unusually high/low call density and checks preservation of coordinates and integer position semantics	cyvcf2 iteration; VCFtools SNP-density analogue
Q2	Record-level allele-shape distribution	REF, ALT; vcfr:ref, vcfr:alt	Summarizes SNV, multiallelic, insertion-shape, deletion-shape, MNV, symbolic/breakend, and no-ALT records; stresses sequence-resolved SV alleles	Explicit cyvcf2 classifier using the same ordered rules
Q3	Biallelic SNV transition/transversion counts	REF, ALT	Reproduces the established Ti/Tv call-set QC statistic and detects allele corruption or case/lexical errors	cyvcf2; VCFtools Ti/Tv analogue
Q4	FILTER state and exact FILTER-value distribution	FILTER; vcfr:filter	Distinguishes PASS, filtering-not-applied (.), and failed filter codes; validates complete FILTER lexical preservation	bcftools query for exact FILTER text; cyvcf2 fallback
Q5	Per-sample genotype classes and call rate	Sample IDs and GT; sample-call and FORMAT-value links	Measures missingness and genotype composition per sample; directly stresses deep sample linkage in 32- and 2,504-sample files	cyvcf2 raw genotype allele indexes
Q6	GT-derived alternate-allele count/number distribution	Single-ALT records, sample GT	Recomputes AC and AN from complete haploid/diploid genotypes instead of trusting INFO tags; tests aggregation across all calls at a site	cyvcf2, with optional bcftools fill-tags cross-check

Docker 28.2.2, Python 3.12.3. The current-version campaign reported in Section 3 was executed separately, on two identically specified hosts, and its full environment, tool commits and resolved container image digests are given in Section 3.1. Every cell of that campaign records its own host, argv, tool commit and image digest, so no result in Section 3 depends on the environment description above.

2.4 Semantic-equivalence and bioinformatic query workload

Syntax and triple-count checks establish that an output is readable, but they do not establish that biologically relevant VCF content can be recovered from the RDF graph. We therefore define six paired operations for independent execution over (i) one VCF and its corresponding RDF representation and (ii) the source VCF with a VCF-native implementation. The workload is summarized in Table 3. It is grounded in standard VCF inspection and quality-control operations exposed by VCFtools, bcftools, and programmatic VCF libraries [6, 45, 46].

Q1–Q4 apply to every file because they use fixed VCF fields represented by the current mapping. Q5–Q6 apply only when the source contains sample columns and a

GT FORMAT field; an empty result is accepted only after header preflight confirms that GT is genuinely absent. GRCh37 and GRCh38 datasets are evaluated separately, and all RDF partitions belonging to one VCF are treated as one logical source.

For raw N-Triples, the queries are executed with the local-file Comunica package `@comunica/query-sparql-file` [47, 48]. For HDT, the same SPARQL text can be evaluated with `@comunica/query-sparql-hdt`, which accepts an `hdt@path/to/file.hdt` source [49]. COTTAS can be evaluated through pycottas' triple-pattern interface or its RDFLib-compatible SPARQL integration [30, 34]; whether it is included in the cross-engine performance table is treated as a separate experimental decision because its execution stack differs from Comunica. On the VCF side, cyvcf2 provides the primary programmatic oracle, while bcftools query preserves the exact FILTER token needed for Q4 [6, 46].

The primary correctness contract is exact equality of normalized integer counts, not approximate biological plausibility. Ratios such as Ti/Tv, call rate, and allele frequency are derived only after their underlying counts agree. The comparator preserves contig and sample identifiers, distinguishes PASS from ., applies the same rules to phased/unphased and missing genotypes, and emits missing/extra/differing rows for localization. Preflight checks additionally validate N-Triples syntax, one-to-one cardinality of core record properties, position datatypes, source/output hashes, tool versions, and record/triple counts.

This workload establishes *representation equivalence* for the stated fields: a SPARQL engine and a conventional VCF parser should derive the same deterministic summaries from two representations of the same source. It does not establish that the source calls are biologically correct, normalized, left-aligned, or concordant with a truth set. The GIAB inputs can support a later reference-aware truth-concordance layer, but that requires confident regions and normalization beyond the present semantic-equivalence test.

[AUTHOR QUERY: The attached testing specification reports that QUAL is retained in the intermediate records TSV but is not emitted by the audited default RML mapping, and that INFO remains available primarily as infoRaw. Please verify the current dev mapping. If QUAL or structured INFO/FORMAT keys have since been added, update this section, the capability table, and the query suite; otherwise do not claim complete support for QUAL.]

This workload is no longer a specification. The query files, cyvcf2 oracle, normalizer and comparator are committed and were executed 62 times across the campaign; Section 3.5 reports the outcomes. Two corrections to the description above follow from what was actually run. First, the executed suite is thirteen questions, not six: Q1–Q6 are the bioinformatic operations of Table 3, and Q7–Q13 add file-metadata, header-line, predicate-census and class-census queries plus three digest queries over record, INFO and FORMAT values, the last three being the queries that detect value-level corruption rather than count-level corruption. Fourteen graph-integrity preflight queries run ahead of them. Second, COTTAS was included in the cross-engine comparison rather than treated as a separate experimental decision, and that inclusion produced a reportable negative result (Section 3.9).

Table 4 Conversion statistics for the original six-file legacy baseline. Sizes are decimal MB. These values should not be interpreted as measurements of the expanded ten-file corpus.

Input VCF	Input .vcf.gz size (MB)	Wall time (s)	Max RSS (MB)	Output raw RDF size (MB)	# triples
0G00R_HG002	387.9	621.0	1,602.4	31,181.3	167,958,298
60820188474283	1,054.8	719.0	1,674.0	35,408.1	185,443,019
NG131FQA1I	1,215.5	630.7	2,027.3	25,969.3	147,710,127
60820188475559	1,233.0	872.2	1,546.3	37,955.6	198,039,339
NB72462M	1,337.0	700.2	1,878.5	26,764.6	155,532,530
NG1N86S6FC	2,076.8	1,693.5	1,943.2	77,729.7	457,412,987

3 Results

Scope of the currently populated results. The numerical values in Tables 4 and 5 come from the original six-file evaluation and the earlier batch-oriented compression implementation. They are retained here as a legacy baseline so that the completed work is not silently conflated with the redesigned storage pipeline. The current one-chunk-at-a-time HDT/COTTAS implementation, the four added files, and the paired SPARQL/VCF-parser workload require new result manifests before numerical claims can be updated. Strategy (2) is adopted. Tables 4 and 5 are retained as an explicitly labelled legacy baseline on the superseded batch-oriented implementation, and every current-version measurement appears in Sections 3.1–3.9 under its own tables. No table mixes the two. One comparison across the boundary is worth stating because it reverses a legacy conclusion: the legacy HDT figures compared HDT against *uncompressed* N-Triples and therefore showed a 92–95% reduction, whereas the current pipeline’s HDT is 1.36–1.82× larger than the gzip-framed N-Triples it is built from (Table 8). The two statements are consistent; only the baseline differs, and the legacy one should not be read as a compression claim against a packaged artifact.

VCF TSV Intermediate Generation (legacy baseline). VCF inputs were first transformed into TSV intermediates to provide a compatible logical source for RMLStreamer and enable consistent field-level extraction during mapping. Across all six datasets, TSV generation completed in 11.9–47.6 seconds with peak RSS of only 4.4–5.4 MB. Each input produced three TSV artifacts (records, header lines, and file metadata) totaling 554.7–2582.3 MB, corresponding to 1.09×–1.43× the compressed VCF input size but only 1.78–5.44% of the final RDF output size. Relative to RDF conversion, TSV generation accounted for just 0.7–7.1% of conversion wall time, indicating minimal computational overhead while providing a stable logical source for declarative mapping execution.

RDF Conversion (legacy baseline). RMLStreamer conversions completed successfully with wall-clock times of 621.0–1693.5 seconds (10.4–28.2 minutes) and peak RSS of 1546.3–2027.3 MB. Each dataset produced eight partitioned RDF outputs with well-balanced triple counts (e.g., 20.91–21.08 million triples per partition for 0G00R_HG002), indicating stable parallel execution.

VCF-RDFizer was executed in *batch* output mode, which preserves partitioned RDF outputs for compression. Reported compression runtimes therefore represent

Table 5 Compression statistics for the original six-file, batch-oriented legacy implementation. Times are wall-clock seconds; sizes are decimal MB. These HDT values predate the current one-chunk-at-a-time HDT/COTTAS pipeline and require a current-version rerun for direct tool claims.

Input file	Time (s)					Size (MB)				
	HDT	Gzip	Brotli	HDT+Gz	HDT+Br	HDT	Gzip	Brotli	HDT+Gz	HDT+Br
OG00R_HG002	70,915.0	192.0	276.0	26.0	64.0	2,102.6	883.2	480.9	215.8	182.0
60820188474283	64,720.0	237.0	348.0	40.0	96.0	1,887.4	1,189.2	731.4	357.6	291.6
NG131FQA1T	92,266.0	183.0	328.0	38.0	106.0	1,876.3	974.2	614.0	320.0	293.2
60820188475559	87,931.0	249.0	366.0	40.0	99.3	2,020.6	1,271.7	775.8	371.0	318.9
NE72462W	91,409.0	204.0	350.0	40.0	117.0	1,975.9	1,032.6	652.8	349.6	311.5
NG1N86S6FC	627,977.0	577.0	868.0	87.0	195.7	5,359.7	2,800.3	1,592.7	682.1	682.4

dataset-level totals across all partition files rather than the cost of compressing a single aggregated RDF file.

Table 4 summarizes conversion results for six VCF datasets. Inputs ranged from 387.9–2076.8 MB and produced 25,969.3–77,729.7 MB of RDF, corresponding to semantic expansion factors of $20.0\times$ – $80.4\times$ relative to compressed VCF input. Outputs contained 147.71–457.41 million triples, demonstrating that the workflow scales across substantially different dataset sizes. Overall, conversion remained tractable on commodity workstation hardware, although memory usage was non-trivial. Under this batch configuration, general-purpose compression required minutes, whereas HDT generation dominated end-to-end runtime.

RDF Output Compression (legacy baseline). Compression results are summarized in Table 5. All methods substantially reduced RDF size but differed in compression ratio, runtime, and memory requirements.

HDT Compression (legacy baseline). HDT reduced RDF outputs to 1876.3–5359.7 MB (92.62–94.68% reduction). However, it required the longest runtimes (64720.0–627977.0 seconds; 18.0–174.4 hours) and the highest memory usage (2566.1–6447.8 MB RSS). Across datasets, HDT accounted for 97.3% of total wall time. Runtime varied by 563257.0 seconds (156.5 hours), corresponding to a $9.7\times$ increase between datasets *60820188474283* and *NG1N86S6FC*; the latter alone contributed 60.7% of total HDT processing time. User and wall times were nearly identical (99.9%), indicating a strongly CPU-bound stage consistent with HDT dictionary and index construction overhead.

Gzip Compression. Gzip reduced RDF size to 883.2–2800.3 MB (96.14–97.17% reduction) with runtimes of 183.0–577.0 seconds and peak RSS of ~ 4 MB. These results confirm gzip as a fast, low-memory archival compression baseline.

Brotli Compression. Brotli produced smaller outputs than gzip for all datasets (480.9–1592.7 MB; 97.56–98.46% reduction) but required longer runtimes (276.0–868.0 seconds; 4.6–14.5 minutes). Peak RSS remained modest at 47.0–51.1 MB, indicating that the additional cost is primarily CPU time. Brotli therefore provides improved compression ratios with moderate runtime overhead while maintaining low memory requirements.

Compound Compression. Applying general-purpose compression after HDT produced the smallest artifacts. HDT+gzip reduced outputs to 215.8–682.1 MB (98.69–99.31% reduction) with an additional 26.0–87.0 seconds of runtime and negligible memory overhead (~ 4 MB RSS). HDT+brotli produced the smallest files overall (182.0–682.4 MB; 98.84–99.42% reduction) with an additional 64.0–195.7 seconds of

runtime and 79.1–110.2 MB peak RSS. Overall, HDT+gzip provided the fastest secondary compression, whereas HDT+brotli consistently achieved the smallest final outputs.

3.1 Scope, provenance and reproducibility of the current campaign

All results in Sections 3.1–3.8 come from a single archived benchmark campaign executed on two physically separate but identically specified hosts (**vcf-bench-1**, **vcf-bench-2**): Ubuntu 24.04.4 LTS, kernel 6.8.0-85-generic, Intel Xeon Silver 4309Y at 2.80 GHz (8 cores), 33.6 GB RAM with swap disabled, a 210 GB SSD, Docker 29.7.2 and Python 3.12.3. Whole experiments, never cells within an experiment, were assigned to a host, so every internal comparison reported below is within-host. Both hosts additionally executed one identical calibration cell before their share: the conversion phase took 29 s on both and produced byte-identical artifacts, and the validation phase differed by 3.7% (324 s versus 336 s), which bounds the residual cross-experiment hardware term.

The campaign comprises 134 recorded cells over 92.4 h of measured wall time (Table 6), plus 12.7 h of runs killed for non-termination, 11.2 h of superseded runs retained for audit, and 0.4 h of calibration. Four tool commits were used — **025fb7d**, **be658a2**, **a3679e1** and **20d2cbb** — and every cell records its own commit, argv, host and resolved container image digest. Because both hosts independently built an image tagged **vcf-rdfizer:local-025fb7d**, that tag denotes different bits on the two machines (**sha256:b645120b...** on **vcf-bench-1**, **sha256:0082fe2c...** on **vcf-bench-2**); the digests, not the tags, are the provenance record. The tool under test is a development branch rather than a release tag, which is stated here rather than concealed.

[AUTHOR QUERY: The testing environment above supersedes the legacy workstation description in Section 2.3. Please confirm you wish to report the development-branch commits rather than delaying publication until a tagged release, and state the intended release tag in the Data/Code availability statements.]

3.2 Does the tool convert every input it is intended to?

Declared VCF versions. Six single-purpose fixtures covering VCFv4.1 through VCFv4.5, plus one file with no **##fileformat** declaration, all converted with exit code 0. The generated RML mapping for each version is byte-identical apart from the asserted file class (**vcfc:VCF41File** through **vcfc:VCF45File**), which locates version awareness in the typed file resource rather than in field-level rules. Field handling is therefore data-driven: the VCFv4.5 fixture’s local-allele **FORMAT** keys (**LAA**, **LAD**) and the VCFv4.4 fixture’s phase-set key (**PS**) are carried through as ordinary **FORMAT** entries, raising the per-record triple yield from 210 (4.4) to 252.5 (4.5) on otherwise matched two-record fixtures. This is preservation, not interpretation: nothing in the graph resolves a local allele index to a global allele, so VCFv4.4/4.5 support should

Table 6 The archived current-version campaign. “Non-zero exit” counts the wrapper’s exit status; Section 3.5 shows that two of the three non-zero cells nevertheless produced a complete graph and failed only at the validation stage. “Guard-skipped” cells are recorded refusals with a stated reason, not missing data.

Experiment	Cells	OK	Skip	Exit $\neq$ 0	Host	Commit	Hours	What it answers
Storage mode	8	8	0	0	b2	025fb7d	13.06	Peak workspace, paired
Sample representation	30	29	1	0	b2	025fb7d be658a2	18.75	Condensed vs expanded, 1–2504 samples
Records scaling	10	10	0	0	b1	025fb7d	10.87	Records ladder and slopes
Corpus breadth	10	9	1	0	b2	be658a2	24.90	Ten real cohorts, one whole
Representation axes	20	20	0	0	b1	be658a2	4.89	INFO and header cost; VCF 4.1–4.5
Robustness	8	8	0	0	b1	be658a2	0.25	Determinism, round trip, faults
Awkward inputs	16	11	2	3	b1	a3679e1	0.09	Eleven difficult VCFs; linking
Feasibility	9	9	0	0	b1	20d2cbb	9.03	Memory ceiling $\times$ configuration
Covering set	10	10	0	0	b1	a3679e1	0.33	Every option value and pair
Modes	7	7	0	0	b1	be658a2	0.02	Phase A/B separation
Query cost	6	6	0	0	b1	20d2cbb	10.21	SPARQL vs cyvcf2 on identical work
Total	134	127	4	3			92.41	

be described as lexical carry-through pending a dedicated local-allele semantics layer. The undeclared-version file also converted, via the documented fallback.

Real cohorts. Nine of ten corpus files converted end to end (Table 8). The tenth, the 2,504-sample 1000 Genomes chromosome-20 call set, was refused by an explicit guard in the *expanded* sample representation only, with the recorded reason that 2,504 sample columns exceed the configured maximum; the same file converts under the *condensed* representation, and Section 3.3 quantifies exactly why the expanded arm cannot be run on that input at this disk budget. Nine of the ten corpus files were truncated to their first 250,000 records while retaining full headers, so the corpus claim is feature coverage across ten cohorts with one file (HG005.GRCh38, 2.36 GB uncompressed) converted whole; it is not a claim that ten whole cohorts were converted.

Difficult inputs. Eleven adversarial fixtures were run (Table 7; fixture files are named `awkward_*` in the testing repository). Eight converted and passed validation, including CRLF line endings, symbolic alleles, breakend ALT syntax, ploidy 4 and 3 in the same file, half-calls and haploid calls, missing values in every optional field, header keys declared but never used, and the three-way `PASS/. /failed-filter` distinction. Of the three cells with a non-zero exit, only one is a refusal to convert: the truncated gzip stream, rejected in 0.58s at the decompression stage with a clear diagnostic, which is the intended behaviour. The other two produced a complete graph and an HDT

Table 7 Difficult-input fixtures. Both “converted” and “refused with a clear diagnostic” are acceptable outcomes; a crash or a silently wrong graph is not, and neither occurred. Triple counts are for the complete graph the run produced.

Fixture	Outcome	Triples	Validation	Note
CRLF line endings	converted	262	PASS	No stray <code>\r</code> in the last field’s literal
Symbolic alleles	converted	453	PASS	<code></code> not treated as a base string
Breakend ALT	converted	417	PASS	Breakend ALT syntax preserved
Mixed ploidy	converted	350	PASS	Ploidy 4 then 3 followed the data
Half and haploid calls	converted	446	PASS	Four genotype cases stayed distinct
Missing values	converted	321	PASS	Missing QUAL, FILTER, INFO and FORMAT absent from the graph
FILTER variants	converted	486	PASS	PASS, . and a failed code are three states
Declared FORMAT unused	converted	293	PASS	Header declaration represented although never used
Zero data records	converted; validation failed	181	blocked by preflight	Header-declared sample not emitted; fixed and re-verified since (Section 3.10)
Sites only	converted; validation inconclusive	245	execution failed	The VCF-side oracle could not read the fixture; see Section 3.5
Truncated gzip	refused at decompression	—	not reached	Refused in 0.58s with a decompression diagnostic; intended

artifact and then failed at the tool’s own validation stage — which is the validator working as designed, and is discussed in Section 3.5.

3.3 Scaling behaviour along the two axes that matter

Records. A four-rung ladder derived from HG005_GRCh38 (10 k, 100 k, 1 M records, then the whole 2.36 GB file) was run in triplicate at the first three rungs. Triple production is exactly linear in records: the fitted exponent is +0.999 (95% CI [+0.991, +1.007], $R^2 = 1.0000$), and the per-record yield is constant at 170.9–171.9 triples. Final artifact bytes scale at +1.033 and peak workspace at +1.021; end-to-end wall time scales at +1.061 from 10 k to 1 M records (41.3 s, 424.0 s, 5,479.3 s; replicate sd ≤ 6.8 s). Peak resident memory is effectively flat at +0.103, staying between 1.0 and 1.7 GB across a hundredfold change in input — the streaming design is memory-bounded, and it is disk, not RAM, that scales with the data. Notably, the RMLStreamer mapping stage itself accounts for only 12.9–40.2 s of that time; end-to-end cost is dominated by the downstream queryable-representation build.

Samples. The same 10,000 variant records were re-emitted against a sample ladder of 1, 4, 16, 64, 256, 1024 and 2504 columns, holding variant content fixed. The two sample representations separate exactly as the semantic model predicts: condensed triple production has a fitted exponent of +0.001 (95% CI [+0.000, +0.002]) — 1,439,503 triples at one sample and 1,452,018 at 2,504, a 0.9% increase over a 2,504-fold change

— while expanded production has an exponent of +0.795 (95% CI [+0.649, +0.940]), reaching 627,372,018 triples, 432 $\times$ the condensed graph for the same variants. End-to-end wall time follows: exponent +0.194 condensed versus +0.800 expanded (236 s versus 14,527 s at 2,504 samples), and the stored artifact grows from 67 MB to 8.77 GB. Extrapolating the expanded representation to the full 1,812,841-record chromosome-20 call set gives roughly 2 TB against a 189 GB volume, which is the measured basis for the guard described in Section 3.2. Condensed encoding is therefore not a convenience option but the only tractable setting at cohort scale, and the manuscript should say so.

Storage mode. The plain and space-optimized storage modes were run paired on two inputs. They are indistinguishable in output — identical triple counts (17,098,746 and 269,272,677) and identical final artifact bytes — and differ only in transient disk. Peak host workspace falls by 7.83 $\times$ (3.047 GB to 0.389 GB) on the 100 k-record input and by 7.32 $\times$ (43.73 GB to 5.98 GB) on the larger one, at a time cost of +4.0% and +3.1% end to end. A pre-registered two-one-sided-test style equivalence check on the mapping stage at a $\pm 10\%$ margin returns +0.2% with a 90% bootstrap CI of $[-1.6\%, +2.6\%]$, i.e. equivalent, on the replicated input; the single-replicate larger input is indeterminate by construction. Because both modes emit the same triples, a table of *final* sizes would show no difference and would misrepresent the redesign; peak workspace is the number that carries the claim.

3.4 Constructing queryable representations

Table 8 replaces the placeholder reserved in the previous draft. Every corpus cell was run with `--rdf-storage-mode space-optimized`, `--hdt-strategy partitioned` and `--representations hdt,cottas`, so HDT and COTTAS were built from the same graph in the same run.

Two results stand out. First, COTTAS is consistently the smallest artifact: 0.37–0.55 $\times$ the size of the stored gzip-framed N-Triples, and 0.29–0.36 $\times$ the size of HDT built from the same graph. Second, uncompressed HDT is *larger* than the gzip-framed N-Triples it was built from, by 1.36–1.82 $\times$. HDT’s value here is therefore indexed triple-pattern access, not compactness; the legacy Section 3 figures, in which HDT appeared as a strong compressor, compared it against uncompressed N-Triples and are not comparable to these. Construction cost runs the other way: COTTAS takes 1.5–2.8 $\times$ longer to build than HDT on the same graph (28,502 s versus 19,480 s on the whole HG005_GRCh38).

Peak workspace, the quantity the storage redesign targets, is 5.7–32.0 $\times$ the uncompressed input across the corpus, with the whole-file cell peaking at 16.23 GB against a 2.36 GB input while producing a 10.88 GB final tree. The whole-file conversion completed in 14.83 h end to end, of which the RMLStreamer mapping stage was 127.9 s.

3.5 Does the tool validate the graphs it produces?

The tool ships a validation stage that executes SPARQL against the graph it just wrote and compares every answer with an independently computed VCF-side result from

Table 8 Current-version queryable-representation construction, one run per file. Nine files are truncated to their first 250,000 records with headers retained; HG005_GRCh38 is the whole file. “N-Triples” is the stored gzip-framed artifact that space-optimized mode writes. “Decode” reports the tool’s native streamed decode and source-versus-decoded triple-count check for both representations.

Dataset	Input (MB)	Triples (M)	N-Tr. (MB)	HDT (MB)	COTTAS (MB)	HDT (min)	COTTAS (min)	Peak (GB)	Dec.
HG002 [†]	13.8	20.52	87.3	158.8	44.7	9.2	28.5	0.44	✓
60820188474283 [†]	68.4	72.13	334.4	484.7	179.9	23.7	65.8	1.53	✓
60820188475559 [†]	80.2	79.65	371.7	547.1	205.0	25.9	70.4	1.73	✓
NB72462M [†]	67.0	59.96	283.5	413.7	153.0	20.3	45.7	1.31	✓
NG131FQA1I [†]	63.8	58.23	276.7	403.7	148.6	20.3	46.9	1.27	✓
NG1N86S6FC [†]	28.1	30.52	146.9	222.7	75.6	12.1	23.3	0.67	✓
HG004 [†]	173.3	40.83	181.3	278.0	93.7	14.6	32.9	0.99	✓
HGSVC2 [†]	145.5	96.75	556.0	754.6	204.1	31.8	69.8	2.31	✓
HG005 (whole)	2359.6	657.43	2885.2	4630.3	1503.9	324.7	475.0	16.23	✓

1000G.phase3_chr20 — guard-skipped in *expanded* mode (2,504 samples); Section 3.3

[†]First 250,000 records, headers retained. Input size is the uncompressed stream as the tool measures it, not the distributed .vcf.gz. Dataset labels are abbreviated; full names are in Table 2. End-to-end wall time per file ranged from 0.67 h to 14.83 h.

cycvf2, with bcftools supplying exact FILTER text. Across the campaign this stage ran 62 times over 134 cells, on N-Triples, HDT and COTTAS artifacts, under four SPARQL engines (Comunica, QLever, an HDT-backed engine, and native pycottas). Table 9 summarizes what that produced. The paired query workload as executed is thirteen questions, not the six defined in Section 2.4: Q1–Q6 are the bioinformatic operations of Table 3, and Q7–Q13 add file-metadata, header-line, predicate and class censuses and three digest queries over record, INFO and FORMAT values. Fourteen further graph-integrity preflight queries run before them.

Of 780 paired query comparisons, 772 returned exact equality, 4 returned a verified not-applicable (the sites-only fixture genuinely has no genotypes), and 4 were mismatches — all four on the same zero-record fixture, discussed below. All 904 cross-check invariants passed. Engine agreement was recorded as true in all 60 completed validations, including the nine in which all four engines answered independently. The tool’s own native decode check — stream the built representation back and compare source and decoded triple counts — passed 142 times out of 142 across 112 HDT and 30 COTTAS artifacts. Syntactic validation by rapper passed on every artifact.

Three graph-stability properties were tested directly rather than assumed. Two conversions of the same input produced byte-identical N-Triples (SHA-256 661578e7..., 17,098,746 triples, zero duplicate lines). A compress-then-decompress round trip reproduced that same digest exactly. Rebuilding the index over an already-indexed artifact left its digest unchanged.

How much would this validation catch? A mutation-score experiment injected 113 targeted corruptions into the graph — wrong allele, wrong coordinate, wrong filter token, wrong file metadata, blank nodes, flipped phasing, corrupted sample index — and measured how many the validation suite detects. It detected 96, a score of 0.850. The 17 undetected instances fall into ten classes, and the report names each one and

why: allele kind and allele value are counted but never compared against the raw ALT column; header attribute values and the derived contig count are counted but not read; record index, sample index, called allele and phasing status have no oracle yet. This is the most useful single number in the paper for the validation question, because it converts “we validate” into a measured detection rate with a named gap list.

What the validation caught in this campaign. One genuine defect surfaced, and one apparent defect turned out to be the check rather than the graph. First, on a header-only VCF that declares a sample column but contains zero data records, the graph omits the `vcfc:SampleSet` and `vcfc:VCFSample` resources the header declares; Q9 and Q10 flagged the two missing predicates and two missing classes, and the run exited non-zero rather than reporting success on a silently incomplete graph. Second, and separately, the sites-only fixture returned `EXECUTION_FAILED` not because the graph is wrong but because the VCF-side oracle could not parse the fixture at all — `bcftools` rejects a `FORMAT` column declared with no sample columns, which is itself outside the specification. That case is a fixture and oracle limitation, and the conversion produced 245 triples and a valid HDT artifact.

One reported anomaly was a false positive. On the real 100,000-record HG005_GRCh38 input the missing-token conformance check returned 20 rows, which reads as 20 missing values emitted as the literal `"."` rather than omitted from the graph. It is not. The published shapes admit a bare dot on exactly two predicates, and both were among those 20: `vcfc:fieldNumber`, whose lexical space is `^([0-9]+|A|R|G|L|A|L|R|L|G|P|M|.|.))$` because `Number=.` is VCF’s variable-cardinality token and therefore a *declared arity*, and `vcfc:genotypeString`, where a fully missing call is literally `"."`. Neither carries `vcfc:Null` because neither is a missing value. The check was over-broad rather than the graph non-conformant; it has since been narrowed to exclude those two predicates, in both the sample query and its count. We report this because the distinction is the whole point of a validation suite: a check that flags conformant data is a defect in the check, and an aggregate pass rate that hides one is worth less than a named correction.

3.6 Retrieval cost: SPARQL against a VCF parser on identical work

Table 10 replaces the placeholder reserved in the previous draft and reports the paired retrieval experiment. The protocol, fixed before the run: three replicates; a cold start per cell; identical question scope on both sides; `cyvcf2` as the VCF-side oracle with `bcftools query` for exact `FILTER` text; engine setup time reported separately from query time; and representation-build time excluded from both columns and reported instead in Section 3.4. Result equality was verified before any timing was compared — every cell in Table 10 is a `PASS`.

The dominant finding is that the artifact barely matters and the engine matters enormously. On the 17.1M-triple graph, per-query times against N-Triples, HDT and COTTAS differ by under 5% for the same engine, because each engine materializes what it needs. Across engines on the 0.96M-triple graph, the same thirteen questions cost 1.1 s under QLever, 23.2 s under Comunica, 44.8 s under the HDT-backed engine

Table 9 Validation evidence produced by the campaign. Every count is over archived report files, not over intentions.

Validation layer	Instances	Outcome	Interpretation
N-Triples syntax (rapper)	62 artifacts	62 PASS	The serialization parses
Graph-integrity preflight (9 checks)	540 checks	529 PASS, 9 expected, 2 FAIL	Blank nodes, duplicate triples, empty values, POS datatype, record cardinality, missing-token conformance, GT inventory
Paired SPARQL vs cyvcf2 (Q1–Q13)	780 comparisons	772 PASS, 4 n/a-verified, 4 MISMATCH	Exact equality of normalized counts, not approximate plausibility
Cross-check invariants	904 assertions	904 PASS	E.g. transitions + transversions = biallelic SNVs
Cross-engine agreement	60 validations (9 with 4 engines)	agreement in all 60	Comunica, QLever, HDT, native pycottas
Native decode and triple count	142 artifacts	142 PASS	112 HDT, 30 COTTAS; source = decoded
Determinism (two runs)	1 pair	identical SHA-256	17,098,746 triples, 0 duplicate lines
Compress and decompress	1 pair	identical SHA-256	Lossless through the packaging path
Index idempotence	1 pair	identical SHA-256	Re-indexing does not perturb the artifact
Mutation score	113 injected faults	96 detected (0.850)	10 named undetected classes
SHACL shape validation	0	not exercised	No run in this campaign enabled it. Since measured: the default profile costs 1.7 s per 2,000 triples but checks cardinality only; the two profiles that check uniqueness and value agreement cost 33.7 s and 73.2 s on that graph (Section 3.9)

and 1,402 s under native pycottas — a spread of three orders of magnitude on identical work over identical data.

The comparison with the VCF parser depends entirely on how the question is asked, and the paper should state both readings. For a *single* question against the 17.1M-triple graph, cyvcf2 must scan the VCF and costs 11.5 s regardless of which question it is, whereas QLever answers in 0.005–4.66 s: SPARQL is $2.5\times$ to $2300\times$ faster per question. For the *whole thirteen-question batch*, cyvcf2 answers all of them in one 12.4 s pass, while QLever needs 17.1 s of query time plus 22.5 s of one-time setup. The RDF path wins on selective, repeated and ad hoc interrogation; the parser wins on a single exhaustive sweep. Neither statement alone is honest, and neither includes the conversion cost in Section 3.4, which the graph must pay once before any of this applies.

Table 10 Paired per-query retrieval on the 17.1M-triple HG005_GRCh38 graph (100,000 records), mean of three replicates. Every comparison returned PASS before any timing was compared. The cyvcf2 column is the cost of answering that one question from the VCF, which requires a scan whichever question it is. QLever setup, 22.5 s, is one-time and excluded; representation-build cost is in Section 3.4.

ID	Operation	QLever (s)	cyvcf2 (s)	Speed-up
Q1	Record density by contig and 1-Mb window	0.048	11.50	240×
Q2	Record-level allele-shape distribution	0.526	11.50	22×
Q3	Biallelic SNV transition/transversion	0.105	11.50	110×
Q4	FILTER state and exact value distribution	0.230	11.50	50×
Q5	Per-sample genotype classes and call rate	1.762	12.25	7×
Q6	GT-derived alternate-allele count/number	3.215	12.25	4×
Q7	File metadata	0.008	11.50	1400×
Q8	Header-line census	0.005	11.50	2300×
Q9	Predicate census	0.201	11.50	57×
Q10	Class census	0.783	11.50	15×
Q11	Record digest	1.764	11.50	7×
Q12	INFO value digest	4.681	11.50	2.5×
Q13	FORMAT value digest	3.783	12.25	3.2×
All thirteen questions		17.11	12.41	0.7×

Table 11 Cost of the same thirteen questions across engines and artifacts. Artifact choice changes query time by under 5% for a given engine; engine choice spans three orders of magnitude. Setup is one-time per artifact and reported separately.

Graph	Engine	Q1–Q13 (s)	Setup (s)	Note
0.96M triples	QLever	1.09	2.07	@comunica/ query-sparql-file
0.96M triples	Comunica	23.30	12.36	
0.96M triples	HDT-backed	44.83	10.80	
0.96M triples	native pycottas	1402.37	29.52	Triple-pattern interface over the columnar store
17.1M triples	QLever, N-Triples	17.11	22.50	VCF-side oracle, single scan
17.1M triples	QLever, HDT	17.16	22.68	
17.1M triples	QLever, COTTAS	16.97	22.74	
17.1M triples	cyvcf2, one pass	12.41	—	

3.7 Do the proposed linking approaches work?

The tool exposes external linking as a separate `--mode link` stage with three tiers, and all three were exercised as far as the offline campaign allowed (Table 12). The result is a working mechanism on thin evidence, and the manuscript should claim exactly that.

Tier 1, identifier rewriting, works and is cheap: on an 86-record graph the `rsid-dbsnp` linker emitted 86 `linking:sameVariantAs` triples to `identifiers.org` dbSNP IRIs in 2.2ms with no network access, no skipped records and a recorded manifest digest. But tier 1 is by construction a syntactic rewrite of an identifier that is already in the ID column; it demonstrates plumbing and provenance capture, not entity resolution, and the fixture’s identifiers are synthetic.

Table 12 Linking tiers as exercised. Input is the same 86-record graph in every case.

Tier	Linker	Keys	Links	Time (s)	Network	Status and limitation
1	rsid-dbsnp	86	86	0.0022	none	Works; an identifier rewrite, not entity resolution; synthetic rsIDs
2	gene-demo	86	0	0.0019	none	Executes and records the reference digest; the bundled demo interval set does not overlap the fixture
3	rsid-ensembl	—	—	—	required	Not run; the campaign was deliberately offline

Tier 2, coordinate-interval lookup against a bundled reference, ran successfully and produced *zero* links from the same 86 records. The run is well-formed — it records the reference digest, the GRCh38 assembly, 86 unique keys and no skipped records — but the bundled demonstration gene set does not overlap the fixture’s coordinates. Tier 2 is therefore demonstrated to execute, not demonstrated to link.

Tier 3, live service resolution against Ensembl, was not run: it requires network access, which the campaign deliberately withheld so that every reported number is reproducible offline. The custom-mapping path was likewise skipped for want of a supplied rules file.

The fair summary is that the linking architecture — tiered linkers, offline operation, per-linker manifest and reference digests, link counts and timings in the run record — is implemented and instrumented, and that its real-world yield is untested. A credible demonstration needs a real annotated cohort against a real dbSNP or Ensembl release with recall and precision reported against a known overlap, and that experiment has not been run.

3.8 Configuration coverage, operating modes and memory feasibility

A ten-row covering set exercised every value of all seven configuration axes — sample representation, INFO representation, storage mode, RDF compression, representations, artifact compression and HDT strategy — with all ten rows exiting 0 and passing validation as the campaign ran it. That verdict did not survive the addition of the shape layer: re-run with SHACL applied by default, three of the ten rows failed, having produced graphs that reference allele resources they never describe. The defect and its fix are reported in Section 3.10, after which all ten rows pass again. The configurations ran, in other words, but the campaign’s validation was not strong enough to say whether what three of them produced was correct. Pairwise coverage is complete except for three combinations the wrapper structurally refuses (`--hdt-strategy single` beside space-optimized storage, beside COTTAS, and beside `hdt,cottas`), so the remaining gaps are specification, not omission. Each of the tool’s operating modes — `tsv`, `compress`, `decompress`, `index` for HDT and for COTTAS, a conversion-only phase and a validation-only phase — ran independently and exited 0, confirming that conversion and validation are separable stages rather than one monolithic command.

A feasibility sweep crossed three memory ceilings (8, 16 and 31 GB) with three configurations on a one-million-record input. All nine cells completed; peak resident memory was 1.37–1.85 GB in every case, two orders of magnitude below even the lowest ceiling, which is consistent with the flat memory scaling in Section 3.3. This experiment carries one caveat that must be reported with it: the ceiling was requested through Docker memory limits but the harness could not confirm that the pinned release forwards them to its container invocations, so these cells establish that the workload is memory-light, not that it survives a genuinely enforced cap.

Finally, the INFO representation axis has a measurable cost: on the structural-variant file, structured INFO produced 96.75M triples against 87.82M for raw INFO (+10.2%) and a 10.2% larger artifact, and on the 100,000-record HG005.GRCh38 input it produced 17.10M against 11.41M (+49.8%). Structured header representation costs almost nothing by comparison (17,098,746 against 17,094,293 triples, +0.03%).

3.9 Limitations established by this campaign

Three limitations are findings of the campaign rather than gaps in it, and are reported here because the evidence for them is archived alongside the positive results.

Native COTTAS querying does not terminate on the condensed genotype encoding. The cross-encoding equivalence experiment was killed twice. In both runs, native pycottas answered the fourteen preflight queries and Q1–Q4 against the condensed encoding and then failed to complete `q05_sample_genotype_counts`, running 41 h in the first attempt and about 10 h in the second at roughly 190% CPU. On the same cell QLever answered all thirteen questions, and the sibling expanded encoding completed every engine in 185–194 min. The defect is specific: native COTTAS querying of the condensed genotype encoding, at a 10,000-record scale. Both killed runs are archived whole, with container logs and process snapshots.

The query timeout does not bound that hang. The second attempt passed `--validation-query-timeout 1800` and `--validation-time-budget 14400` on the command line, and the process snapshot confirms both flags were live on the running invocation. Neither fired. A bound that is documented but not enforced is worse than none, because it invites unattended runs; this should be fixed before release.

The equivalence claim is therefore partial. What did complete is still substantive: on the expanded encoding, all four engines validated all three artifacts (N-Triples, HDT, COTTAS) with full agreement and PASS status, at 12.7 s (QLever), 342.1 s (HDT-backed), 384.7 s (Comunica) and 2,894.2 s (native pycottas) for the twenty-seven-query suite. The condensed arm has no COTTAS-side result, and the paper claims cross-encoding equivalence for the expanded encoding only.

Two further scope limits are worth stating plainly. The derived sample ladders are not valid VCFs — sample columns were cut without recomputing INFO, so AC/AN in the header disagree with the retained genotypes. This is deliberate, because INFO is held fixed as a control across the ladder, but it means the ladder measures structural scaling and not INFO-dependent behaviour. And nine of the ten corpus files are truncated, so the corpus supports a feature-coverage claim over ten cohorts with one file converted whole, not a throughput claim over ten whole cohorts.

3.10 Defects acted on after the campaign

The campaign is archived as run, and none of the numbers above have been restated. The defects it surfaced were then fixed and the fixes re-verified on `vcf-bench-1`, at fixture scale, against a container image built from the same source. We report that separately rather than folding it into the results, because a fix verified on a 1,000-record fixture is not a re-run of a 92-hour campaign.

The zero-record graph is complete. The sample columns a `#CHROM` line declares are now emitted whether or not the file carries data records: the emitters keyed their file IRI on a source name learned from the first data row, so a header-only VCF produced nothing at all. On `awkward_no_records` the graph goes from 181 triples and a non-zero exit to 189 triples and a clean pass, with Q9 and Q10 no longer reporting the six predicates and two classes they flagged, and with the two `NOT_APPLICABLE_VERIFIED` verdicts unchanged — a zero-record file genuinely has no genotypes.

One oracle expectation was wrong in the same case. With the sample set restored, `preflight_sample_gt_inventory` still failed, expecting one `vcfc:sampleId` and finding none. That literal lives on `vcfc:SampleCall`, one per sample *per record*, so a file with no records structurally has none; the expectation, not the graph, was at fault.

The validation bound is now enforced where queries execute. The per-query timeout that did not fire in Section 3.9 could not: the native COTTAS engine queried in-process, so nothing could interrupt it, and no engine ever returned the exit status the bounding logic keyed on. Both are fixed. As an interim, the combination that did not terminate is refused outright with its measurement quoted, and `--validation-engine all` drops that one engine with a warning and validates on the other three rather than failing the run — which is what the killed cells needed.

The shape layer is bundled, and its cost is now known. The profile is shipped with the tool and applied by default, so it no longer requires a separate vocabulary checkout, which is why no run in the campaign enabled it. Enabling it exposed two faults in the validation code itself, both of which would have silently misreported: the verdict keyed on `pyshacl`'s `conforms` flag, which is false for a mere recommendation and so failed every conformant VCF carrying an `INFO` declaration without `Source`; and the violation matcher looked for a report format `pyshacl` does not emit, so it had never counted a violation at all. Results are now classified by severity, and only `sh:Violation` blocks a run.

Measuring the profiles then changed what can be claimed for them, and this is the substantive correction of the three. On a 2,000-triple graph the default profile costs 1.7 s, the consistency profile 33.7 s and the SPARQL profile 73.2 s. The four undetected mutation classes that Section 3.5 attributes to the shape layer — allele value, record index, indexed-value allele and expanded sample index — are covered by the latter two, not by the default: the default constrains `vcfc:sampleIndex` to occur once as an integer and says nothing about whether two samples share a value, while uniqueness is a `sh:spargl` rule that self-joins the graph. So the default is the cheap profile and the expensive pair is an explicit opt-in, gated to small inputs.

With all three profiles enabled, the suite detects every injected corruption. Re-running the mutation experiment on the fixed tool raises the score from 96/113 to **113/113**: 96 corruptions are caught by the comparison queries alone, 17 by the shapes, and no class remains undetected. That closes all ten classes named in Section 3.5, not the four the cost measurement attributed to the shape layer — the constraints reach further than the mapping from mutation class to profile predicted, because a corruption that does not violate its own class’s constraint frequently violates a neighbouring one. Both unmutated controls pass, in both sample representations, so the perfect score is not an artefact of a shape layer that rejects conformant graphs. The determinism control was re-run from the same image and reproduces the campaign digest exactly — 17,098,746 triples, zero duplicates, byte-identical output — which is what licenses reporting a fixture-scale fix beside campaign-scale conversion numbers.

Applying the shape layer by default then exposed a further defect, in the conversion path rather than the validator. Re-running the ten-row covering set on the fixed tool, three rows exited non-zero — exactly the three combining `--info-representation raw` with `--sample-representation expanded`. The allele layer was minted only for the structured INFO representation, on the reasoning that the `Number=A/R/G` value items are what join to the allele resources. The expanded sample layer joins to them too: it emits `vcfc:calledAllele` on every `vcfc:GenotypeAlleleCall`, regardless of the INFO setting. Those configurations therefore produced graphs carrying 1,972 references per 1,000 records to allele resources that were never described, with no `vcfc:alleleIndex` triple anywhere in the file. This is the sharpest evidence the study offers for validating at two layers: the comparison queries count `calledAllele` edges without following the join, so all ten rows exited 0 for the entire campaign, while the shapes reported 5,916 violations the first time they ran.

The repair took two steps, and the second is itself informative. Fixing the emitter — making the allele layer a disjunction over both representation axes rather than a property of the INFO setting — left those rows still failing, now on the census comparison rather than the shapes: the graph had gained the allele triples, and the oracle, which derives its expectation from the same representation flags, still expected them absent. The same rule had been written twice and was wrong in both places. With the oracle reading the disjunction too, the covering set returns to ten rows of ten passing, and the change is confined to exactly the three affected configurations, each gaining 10,846 triples — $3 \times 1,972$ allele literals, 986 each of `hasReferenceAllele`, `hasAltAllele` and `vcfc:chromosome`, and 1,972 `rdf:type` — while the other seven produce byte-identical output. A regression test now asserts, across all four representation combinations, that no graph references an allele it does not describe, and that the oracle expects what the emitter writes.

[AUTHOR QUERY: Section 3.10 now mixes two evidence classes: fixture-scale fixes, and three experiments (08_robustness, 09_awkward_inputs, 11_covering_set) genuinely re-run on the fixed tool. The determinism digest is byte-identical to the campaign, so the conversion-scale results in Sections 3.1–3.4 need no re-run. Decide before submission whether to (1) keep Section 3.10 as the single “since the campaign” subsection it is

now, or (2) fold the re-run experiments back into Sections 3.5 and 3.8 and restate those tables from the fixed code state, leaving only the unmeasured fixes here. Option (2) reads better but requires re-deriving Table 7 and the mutation figures throughout.]

4 Discussion

VCF-RDFizer provides a reproducible pathway for converting heterogeneous VCF datasets into structured RDF representations using a public semantic vocabulary and modular declarative mappings. The original six-file benchmark indicates that conversion was practical for the six baseline inputs: the legacy inputs reported in Table 4 were converted in 10.4–28.2 minutes with peak RSS of approximately 1.5–2.0 GB, while semantic lifting expanded them to tens of gigabytes of N-Triples. General-purpose gzip and Brotli substantially reduced those outputs at low memory cost. The original HDT measurements, however, characterize the superseded batch-oriented build path and should not be used to infer current HDT or COTTAS construction performance.

The current implementation addresses a different operational bottleneck: temporary disk amplification during queryable-representation construction. Incremental reading and one-row-chunk-at-a-time conversion remove the full decompressed RDF duplicate, although final representation parts and balanced-merge workspaces still consume disk. Section 3.3 quantifies that change on paired runs: the two storage modes emit identical triples and identical final artifacts, and differ only in transient disk, which falls by 7.3–7.8 $\times$ for a 3–4% increase in end-to-end time. Peak workspace, not final size, is the quantity that carries the claim, and a final-size comparison would have shown nothing.

Multi-sample behaviour is no longer speculative. Section 3.3 measures it directly on a 1–2504 sample ladder holding variant content fixed, and the two sample representations separate by three orders of magnitude: condensed triple production is flat in sample count (fitted exponent +0.001; a 0.9% increase from 1 to 2,504 samples), whereas expanded production grows at +0.795 and reaches 432 $\times$ the condensed graph at 2,504 samples. The practical consequence is a hard one: extrapolated to the full chromosome-20 call set, the expanded representation would require roughly 2 TB, so it was guarded out and the cohort file was converted in the condensed representation only. Condensed encoding is the operative setting at cohort scale, not a space-saving option.

Table 13 summarizes the implementation and evaluation status supported by the supplied manuscript artifacts. Because the exact release tag, development commit, and active default mapping must be pinned for the final experiments, version-specific and blanket “fully supported” claims are deferred to the individual rows and author queries below.

[AUTHOR QUERY: Resolve the QUAL row against the exact dev mapping commit used for the manuscript. Also confirm whether COTTAS indexes are fixed to spo or user-selectable in the release being evaluated.]

4.1 Semantic Representation of VCF Challenges

Because VCF encodes both biological observations and file-structure semantics, representation in RDF requires a vocabulary that explicitly models the structural elements of the format itself. Existing ontologies capture important aspects of genomic variation, including FALDO for genomic coordinates, the Sequence Ontology (SO) [25] for variant types, GENO for genotype modeling [50], and HERO for clinically oriented genomic semantics [31], but none fully represent the structural semantics of the VCF specification.

To address this gap, we developed the publicly available VCF-RDFizer vocabulary (<https://w3id.org/vcf-rdfizer/vocab#>), which provides a lightweight semantic layer aligned with the structural components of the VCF specification while remaining interoperable with existing genomic ontologies. Future work will extend the vocabulary to better capture heterogeneous INFO and FORMAT field semantics, explicit FORMAT keys, ordered alternate alleles, typed missing values, and richer structural-variant semantics. SHACL constraints can complement the paired queries by checking graph shape, whereas the paired SPARQL/VCF-parser workload tests whether standard summaries can be reproduced from actual data. The completed numerical baseline focused on the original six inputs, but the evaluation corpus now includes two multi-sample files. Their purpose is to test sample linkage and genotype aggregation rather than to imply that multi-sample scalability has already been demonstrated. Moreover, exact agreement with a VCF parser establishes preservation of represented values, not biological correctness of the source calls.

4.2 The Published VCF-RDFizer Software Tool

Transformation Architecture and Performance. RML provides a declarative framework in which semantic intent is separated from procedural parsing logic, allowing transformation rules to be versioned and reused across datasets. In the original six-input baseline, RMLStreamer completed conversion in 10.4–28.2 minutes with peak RSS of approximately 1.5–2.0 GB. These measurements show feasibility for those inputs on the reported workstation; they do not yet establish behavior for the two new multi-sample datasets or the current compression implementation.

The pipeline includes a lightweight preprocessing step that converts VCF files into tabular intermediates compatible with existing RML engines. Because VCF is not yet supported as a native logical source within RML, this step enables stable field extraction and declarative mapping execution. Future extensions to the RML ecosystem could support VCF as a native logical source and eliminate the preprocessing stage.

The baseline demonstrates successful pipeline completion and production of syntactically inspectable RDF, while the paired SPARQL/VCF-parser workload is intended to supply stronger, field-level evidence that the semantic lifting preserves the defined variant and genotype summaries. The substantial expansion in raw RDF size remains an operational limitation and motivates both archival compression and directly queryable compressed representations.

Compression Strategies for Semantic Genomic Data. The legacy results establish gzip as a fast, low-memory archival baseline and Brotli as a stronger general-purpose compressor with moderately greater CPU cost. General-purpose compression remains useful for transfer and preservation, but a `.nt.gz`, `.hdt.gz`, or `.cottas.br` package is not the artifact queried directly; it must first be unpacked.

For compressed RDF that remains queryable, VCF-RDFizer now offers both HDT and COTTAS. HDT combines a dictionary and compressed triple structure with indexed triple-pattern access [29]. COTTAS stores RDF as a Parquet triple table and uses columnar organization and block-range indexes to evaluate patterns over compressed data; its `pycottas` implementation translates RDF access into operations over the columnar representation [30]. These formats provide complementary design points, and the COTTAS paper’s general RDF benchmarks must not be assumed to predict VCF-RDFizer-specific size or query performance.

The earlier draft described balanced HDTCat construction as future work and attributed the COTTAS citation to Jelly. Both statements are superseded. The current pipeline already converts record-safe chunks, merges HDT parts pairwise with HDTCat [33], merges COTTAS parts pairwise with `pycottas`, rebuilds final indexes, and validates each final representation through native streamed decoding and a source/decoded triple-count comparison. Jelly is a separate RDF streaming serialization [51]; it is not the COTTAS output implemented here.

The key storage optimization is that an incrementally decompressed aggregate produces only one raw chunk at a time. This removes the largest temporary duplicate while preserving the ability to build both representations. It does not eliminate the space required by HDT/COTTAS part files, balanced merge rounds, indexes, or DuckDB/converter scratch data. Table 8 now reports final size, build time and peak workspace for both representations across the corpus, and the result is not the one the legacy table suggested. COTTAS is the smaller artifact everywhere, at $0.37\text{--}0.55\times$ the stored gzip-framed N-Triples and $0.29\text{--}0.36\times$ the HDT built from the same graph, while uncompressed HDT is $1.36\text{--}1.82\times$ *larger* than the N-Triples it indexes. HDT’s contribution is indexed triple-pattern access rather than compactness, and the legacy figures in Section 3, which compared HDT against uncompressed N-Triples, are not comparable to these. Construction cost runs the other way: COTTAS takes $1.5\text{--}2.8\times$ longer to build. Neither representation changes query time materially (Section 3.6), so the choice between them is a storage-versus-build-time decision, not a query-performance one.

Tool Availability and Community Impact. Following best practices for research software publication [52], VCF-RDFizer is distributed as open-source software with public version control and synchronized releases across GitHub, PyPI, Conda-Forge, and DockerHub under the MIT License. Public issue tracking, release notes, and a documented maintenance plan support transparency and long-term sustainability.

Queryable Genomic RDF as a Complementary Access Layer. The selected workload illustrates the practical scope of SPARQL even before external annotations are linked. Density windows, allele-shape summaries, Ti/Tv, FILTER distributions, per-sample missingness/genotype classes, and genotype-derived AC/AN are recognizable bioinformatic operations rather than synthetic graph queries. Reproducing

them from RDF tests whether genomic coordinates, alleles, filter semantics, sample identities, FORMAT values, and record-to-sample relationships survive semantic lifting.

The longer-term value of SPARQL is not limited to reproducing operations that VCF-native tools already perform well. Once variant records use stable IRIs and ontology-aligned predicates, a query can combine variant/genotype constraints with linked gene, phenotype, pharmacogenomic, provenance, or policy statements without exporting each relation into a new ad hoc table. Multiple VCF-derived graphs can be queried through the same data model, and Comunica’s source-oriented architecture can query one or more local HDT files through a common SPARQL interface [47, 49]. COTTAS provides another compressed query path through pycottas/RDFLib [30, 34]. These capabilities could support cohort QC across independently produced VCFs, integrated variant prioritization, knowledge-graph enrichment, and governance-aware retrieval using standards such as ODRL [8].

VCF-native tools remain an essential comparator and, for many format-specific operations, are likely to remain the operational baseline. bcftools and cyvcf2 operate directly over VCF/BCF and expose highly optimized parsing, indexing, and genotype access [6, 46]. The RDF pathway adds conversion and representation-build costs in exchange for a generic graph query model and semantic joins. It should therefore be positioned as complementary infrastructure rather than an unconditional replacement for VCF/BCF.

A fair comparison must separate one-time conversion/index construction from repeated query execution; apply identical scopes and allele/genotype rules; record cold-cache conditions, result cardinality, wall time and repetitions; and verify result equality before comparing speed. Section 3.6 follows that protocol, and the answer is conditional rather than directional. On a 17.1M-triple graph, a *single* question costs cyvcf2 11.5s because it must scan the VCF whichever question it is, while QLever answers in 0.005–4.66s — between 2.5× and 2300× faster. For the *whole* thirteen-question batch, cyvcf2 answers all of them in one 12.4s pass against 17.1s of query time plus 22.5s of one-time setup for QLever. The RDF path therefore wins on selective, repeated and ad hoc interrogation and loses on a single exhaustive sweep, and neither reading holds without the other. The artifact makes almost no difference — N-Triples, HDT and COTTAS are within 5% for the same engine — whereas the engine spans three orders of magnitude, from 1.1s under QLever to 1,402s under native pycottas for the same thirteen questions. Engine selection, not representation selection, is the performance decision. None of these figures includes the conversion cost in Section 3.4, which the graph must pay once before any of it applies.

[AUTHOR QUERY: The comparator set is now fixed: cyvcf2 as the programmatic oracle with bcftools query for exact FILTER text, and four SPARQL engines reported side by side including native pycottas. Please confirm whether you also want an indexed-regional-access arm (tabix/CSI), which the present design does not exercise because none of the thirteen questions is region-restricted; adding one would strengthen the parser side of the comparison.]

4.3 Impact and Future Work

This work demonstrates that VCF datasets can be systematically transformed into RDF using declarative RML semantic mappings. More importantly, we show that traditionally static file-based genomic variant data can be reliably converted to structured knowledge graphs, capable of linking, semantic querying, and governance-aware data sharing.

The current-version campaign is complete and archived, and it sets the next agenda rather than closing it. Three concrete items follow directly from Sections 3.5–3.9. First, the validation suite detects 85.0% of injected graph corruptions, and the ten undetected classes are named: allele kind and value, header attribute values, contig count, record index, sample index, called allele and phasing status. That layer is now bundled and applied by default, and with all three profiles enabled the score is 113/113 (Section 3.10), so the gap is closed rather than merely diagnosed. What remains is a cost problem, not a coverage one: the profiles that check uniqueness and value agreement cost roughly sixty times the default one on the same graph, because their constraints self-join it, which confines a perfect score to inputs small enough to afford it. Making that coverage available at cohort scale — by expressing the same uniqueness checks as streaming comparisons over the oracle rather than as `sh:sparql` self-joins — is the concrete next step, and it is the one that would let the campaign’s own conversions be validated as strictly as its fixtures are. Second, native pycottas querying does not terminate on the condensed genotype encoding. The query timeout that failed to bound it has since been fixed and the combination is refused in the interim, but the underlying non-termination is an upstream matter and must be resolved before a release claims cross-encoding query equivalence. Third, the linking architecture is implemented and instrumented but its yield is untested: tier 2 produced zero links against its demonstration reference and tier 3 was never run. A credible linking claim needs a real annotated cohort against a released dbSNP or Ensembl version, with recall and precision reported against a known overlap. The instrumentation for that now exists: each linker records the population of records that produced a join key, the subjects actually linked, and what its assertion rests on — a tier-1 link is an identifier rewrite taken from the source ID column and is not position-verified, which a consumer of `linking:sameVariantAs` needs to know. The denominators are in place; the cohort experiment is not.

Subsequent work will extend semantic coverage of heterogeneous INFO, FORMAT, genotype, header, ordered alternate-allele, and structural-variant constructs; evaluate VCF as a native RML logical source; and develop richer links to clinical and biomedical resources. A separate GIAB truth-concordance experiment could test biological call accuracy, but it must use the appropriate benchmark regions and reference-aware normalization and must remain distinct from representation equivalence.

Ultimately, VCF-RDFizer represents a step toward semantically interoperable genomic data infrastructure in which variant datasets can be compressed, linked, queried, validated, and governed as part of a broader biomedical knowledge-graph ecosystem.

Table 13 Current VCF-RDFizer capability status and evaluation scope. “Supported” refers to the implementation described by the supplied change log/testing specification; “evaluation pending” prevents corpus inclusion from being mistaken for a completed result.

VCF component / capability	Implementation status	Evaluation status	Notes / limitations	
Core record fields (CHROM, POS, ID, REF, ALT, FILTER)	Supported	Validated	Q1–Q4 exact-equal against cyvcf2/bcftools in every completed validation (Section 3.5)	
QUAL	Author confirmation required	Not in core queries	Testing audit reports QUAL in the records TSV but absent from the audited default RDF mapping	
INFO	Partial	Raw preservation testable	Complete INFO string is available as <code>infoRaw</code> ; structured key/value semantics remain limited	
FORMAT and sample/genotype values	Partial	Validated for Q5–Q6; Q13 digest passes	Exact-equal on all applicable artifacts; half-calls, haploid calls and ploidy 4/3 survive conversion (Table 7). The mutation-score gap list (Section 3.5) still includes called allele, sample index and phasing status	
HEADER metadata	Partial	Structural inspection only	Header assertions are represented; complete semantic coverage and declaration-to-use validation remain future work	
Multi-sample conversion	Implemented in model/pipeline	Measured, 1–2504 samples	Condensed is flat in sample count; expanded grows at +0.795 and is not tractable at 2,504 samples (Section 3.3)	
Structural and symbolic/sequence alleles	Lexically supported; semantic typing partial	HGSVC2 converted; Q2 passes	Symbolic and breakend ALT fixtures convert and validate; ALT remains the source lexical field, and allele-value corruption is one of the undetected mutation classes	
Queryable output	HDT	Supported	Measured	112 artifacts, all native decode/count checks passed; 1.36–1.82× the stored N-Triples (Table 8)
Queryable COTTAS output	Supported	Measured, with one defect	30 artifacts, all decode checks passed, smallest artifact throughout; native pycottas querying does not terminate on the condensed encoding (Section 3.9), a combination the tool now refuses	
Gzip/Brotli artifact packaging	Supported	Exercised in the covering set	All values of both compression axes ran and validated (Section 3.8); packaged files must be unpacked before direct querying	
Provenance/policy annotations	Not implemented in conversion pipeline	Not evaluated	RDF permits later linkage, but governance semantics are not generated by the current default mapping	
External linking (tiers 1–3)	Implemented instrumented	Partially exercised	Tier 1 links 86/86 records offline; tier 2 executes but yields zero links on the demonstration reference; tier 3 was not run (Section 3.7)	

Declarations

4.4 Conflicts of interest

None declared.

4.5 Funding

Elias Crum acknowledges support from the Research Foundation – Flanders (FWO) (SB Fellowship 1S27825N). Bart Buelens acknowledges support from ... Ruben Taelman acknowledges support from ... Gokhan Ertaylan acknowledges support from ...

4.6 Data availability

The large VCF inputs are downloaded from their original public providers rather than redistributed in Git. Canonical filenames, source URLs/download commands, experiment manifests, query files, normalized outputs, and comparison reports are maintained at <https://github.com/ecrum19/vcf-rdfizer-testing>. Each reported run should include hashes for the source VCF, all RDF/representation artifacts, and each query.

4.7 Code availability

The VCF-RDFizer source and container workflow are available at <https://github.com/ecrum19/VCF-RDFizer>; testing and paper artifacts are available at <https://github.com/ecrum19/vcf-rdfizer-testing>. A release tag or immutable commit and Docker image digest must be cited for the final experiments.

[AUTHOR QUERY: Replace the development-branch reference with the exact release tag/commit and container digest used for all final results.]

4.8 Author contributions statement

E.C. and G.E. conceived the tool and the experiments. E.C. implemented the conversion workflow and executed the experiments. E.C. and R.T. analyzed the results. E.C. wrote the manuscript draft. E.C., R.T., G.E., and B.B. reviewed and edited the manuscript.

4.9 Acknowledgments

All benchmarking computation was performed using the resources of IDLab at Ghent University.

- Ethics approval and consent to participate
- Consent for publication
- Materials availability

References

- [1] The Variant Call Format Specification (2024). <https://github.com/samtools/hts-specs>
- [2] Li, B., Sangkuhl, K., Keat, K., Whaley, R.M., Woon, M., Verma, S., Dudek, S., Tuteja, S., Verma, A., Whirl-Carrillo, M., Ritchie, M.D., Klein, T.E.: How to run the pharmacogenomics clinical annotation tool (pharmcat). *Clinical Pharmacology & Therapeutics* **113**(5), 1036–1047 (2023) <https://doi.org/10.1002/cpt.2790>
- [3] Souche, E., Beltran, S., Brosens, E., Belmont, J.W., Fossum, M., Riess, O., Gilissen, C., Ardeschirdavani, A., Houge, G., Van Gijn, M., Clayton-Smith, J., Synofzik, M., De Leeuw, N., Deans, Z.C., Dincer, Y., Eck, S.H., Van Der Crabben, S., Balasubramanian, M., Graessner, H., Sturm, M., Firth, H., Ferlini, A., Nabbout, R., De Baere, E., Liehr, T., Macek, M., Matthijs, G., Scheffer, H., Bauer, P., Yntema, H.G., Weiss, M.M.: Recommendations for whole genome sequencing in diagnostics for rare diseases. *European Journal of Human Genetics* **30**(9), 1017–1021 (2022) <https://doi.org/10.1038/s41431-022-01113-x>
- [4] Mitsuhashi, N., Toyo-oka, L., Katayama, T., Kawashima, M., Kawashima, S., Miyazaki, K., Takagi, T.: TogoVar: A comprehensive Japanese genetic variation database. *Human Genome Variation* **9**(1), 44 (2022) <https://doi.org/10.1038/s41439-022-00222-9>
- [5] Van Der Horst, E., Unni, D., Kopmels, F., Armida, J., Touré, V., Franke, W., Crameri, K., Cirillo, E., Österle, S.: Bridging Clinical and Genomic Knowledge: An Extension of the SPHN RDF Schema for Seamless Integration and FAIRification of Omics Data. (2023). <https://doi.org/10.20944/preprints202312.0373.v1>
- [6] Danecek, P., Bonfield, J.K., Liddle, J., Marshall, J., Ohan, V., Pollard, M.O., Whitwham, A., Keane, T., McCarthy, S.A., Davies, R.M., Li, H.: Twelve years of samtools and bcftools. *GigaScience* **10**(2), 008 (2021) <https://doi.org/10.1093/gigascience/giab008>
- [7] World Wide Web Consortium (W3C): LinkedData. <https://www.w3.org/wiki/LinkedData> Accessed 2026-03-04
- [8] Iannella, R., Villata, S.: ODRL Information Model 2.2. W3C (2018). <https://www.w3.org/TR/odrl-model/> Accessed 2025-11-27
- [9] Cyganiak, R., Wood, D., Lanthaler, M.: RDF 1.1: Concepts and Abstract Syntax (2014). <https://www.w3.org/TR/2014/REC-rdf11-concepts-20140225/>
- [10] Hogan, A., Blomqvist, E., Cochez, M., D’Amato, C., Melo, G., Gutierrez, C.,

- Kirrane, S., Labra Gayo, J.E., Navigli, R., Neumaier, S., Ngonga Ngomo, A.-C., Polleres, A., Rashid, S.M., Rula, A., Schmelzeisen, L., Sequeda, J., Staab, S., Zimmermann, A.: Knowledge graphs. *ACM Computing Surveys* **54**(4) (2021) <https://doi.org/10.1145/3447772>
- [11] Belleau, F., Nolin, M.-A., Tourigny, N., Rigault, P., Morissette, J.: Bio2RDF: Towards a mashup to build bioinformatics knowledge systems. *Journal of Biomedical Informatics* **41**(5), 706–716 (2008) <https://doi.org/10.1016/j.jbi.2008.03.004>
- [12] Breit, A., Ott, S., Agibetov, A., Samwald, M.: OpenBioLink: A benchmarking framework for large-scale biomedical link prediction (2019) <https://doi.org/10.48550/ARXIV.1912.04616> . Publisher: arXiv Version Number: 2
- [13] SIB Swiss Institute of Bioinformatics RDF Group Members, Altenhoff, A., Bairoch, A., Bansal, P., Baratin, D., Bastian, F., Bolleman*, J., Bridge, A., Burdet, F., Cramer, K., Dauvillier, J., Dessimoz, C., Gehant, S., Glover, N., Gnodtke, K., Hayes, C., Ibberson, M., Kriventseva, E., Kuznetsov, D., Frédérique, L., Mehl, F., Mendes De Farias*, T., Michel, P.-A., Moretti, S., Morgat, A., Österle, S., Pagni, M., Redaschi, N., Robinson-Rechavi, M., Samarasinghe, K., Sima, A.-C., Szklarczyk, D., Topalov, O., Touré, V., Unni, D., Von Mering, C., Wollbrecht, J., Zahn-Zabal*, M., Zdobnov, E.: The SIB Swiss Institute of Bioinformatics Semantic Web of data. *Nucleic Acids Research* **52**(D1), 44–51 (2024) <https://doi.org/10.1093/nar/gkad902>
- [14] Harris, S., Seaborne, A., Prud’hommeaux, E.: SPARQL 1.1 Query Language (2013). <https://www.w3.org/TR/2013/REC-sparql11-query-20130321/>
- [15] Lindenbaum, P.: JVarkit: java-based utilities for Bioinformatics (2015) <https://doi.org/10.6084/m9.figshare.1425030.v1>
- [16] Penha, E.D.S., Iriabho, E., Dussaq, A., Oliveira, D.M., Almeida, J.S.: Isomorphic semantic mapping of variant call format (VCF2RDF). *Bioinformatics* **33**(4), 547–548 (2017) <https://doi.org/10.1093/bioinformatics/btw652>
- [17] Böhmer, J.K., Janssen, R.: Linked data in practice: An RDF-based approach with SPARQLing Genomics. Zenodo. Version V4 (2020). <https://doi.org/10.5281/zenodo.3903111>
- [18] Garrison, E., Kronenberg, Z.N., Dawson, E.T., Pedersen, B.S., Prins, P.: A spectrum of free software tools for processing the VCF variant call format: vcflib, bio-vcf, cyvcf2, hts-nim and slivar. *PLOS Computational Biology* **18**(5), 1009123 (2022) <https://doi.org/10.1371/journal.pcbi.1009123>
- [19] Bendiken, A., Bonnal, R.J.P., Strozzi, F.: ruby-rdf/rdf-vcf: RDF VCF parser (2025). <https://github.com/ruby-rdf/rdf-vcf> Accessed 2025-03-01
- [20] Bodrug-Schepers, A., Chabane, H., Montoya, G., Redon, R., Gaignard, A.,

- Serrano-Alvarado, P.: Semantic Beacons: a framework to support federated querying over genomic variants and public Knowledge Graphs. CEUR Workshop Proceedings, Barcelona, Spain (2025). <https://hal.science/hal-04908530v2/document> Accessed 2025-11-24
- [21] Janssen, R.: SPARQLing Genomics. Open-source software. Accessed 2026-09-03 (2019). <https://gitlab.com/roelj/sparqling-genomics>
- [22] Prasanna, S., Kumar, A., Rao, D., Simoes, E.J., Rao, P.: A scalable tool for analyzing genomic variants of humans using knowledge graphs and graph machine learning. *Frontiers in Big Data* **7**, 1466391 (2025) <https://doi.org/10.3389/fdata.2024.1466391>
- [23] Wagner, A.H., Babb, L., Alterovitz, G., Baudis, M., Brush, M., Cameron, D.L., Cline, M., Griffith, M., Griffith, O.L., Hunt, S.E., Kreda, D., Lee, J.M., Li, S., Lopez, J., Moyer, E., Nelson, T., Patel, R.Y., Riehle, K., Robinson, P.N., Rynearson, S., Schuilenburg, H., Tsukanov, K., Walsh, B., Konopko, M., Rehm, H.L., Yates, A.D., Freimuth, R.R., Hart, R.K.: The GA4GH Variation Representation Specification: A computational framework for variation representation and federated identification. *Cell Genomics* **1**(2), 100027 (2021) <https://doi.org/10.1016/j.xgen.2021.100027>
- [24] Bolleman, J.T., Mungall, C.J., Strozzi, F., Baran, J., Dumontier, M., Bonnal, R.J.P., Buels, R., Hoehndorf, R., Fujisawa, T., Katayama, T., Cock, P.J.A.: FALDO: a semantic standard for describing the location of nucleotide and protein feature annotation. *Journal of Biomedical Semantics* **7**(1), 39 (2016) <https://doi.org/10.1186/s13326-016-0067-z>
- [25] Eilbeck, K., Lewis, S.E., Mungall, C.J., Yandell, M., Stein, L., Durbin, R., Ashburner, M.: The Sequence Ontology: a tool for the unification of genome annotations. *Genome Biology* **6**(5), 44 (2005) <https://doi.org/10.1186/gb-2005-6-5-r44>
- [26] De Meester, B., Heyvaert, P., Delva, T.: RDF Mapping Language (RML) (2024). <https://rml.io/specs/rml/> Accessed 2025-11-24
- [27] Oo, S.M., Haesendonck, G., De Meester, B., Dimou, A.: RMLStreamer-SISO: An RDF Stream Generator from Streaming Heterogeneous Data. In: Sattler, U., Hogan, A., Keet, M., Presutti, V., Almeida, J.P.A., Takeda, H., Monnin, P., Pirrò, G., d’Amato, C. (eds.) *The Semantic Web – ISWC 2022* vol. 13489, pp. 697–713. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-19433-7_40 . Series Title: Lecture Notes in Computer Science. https://link.springer.com/10.1007/978-3-031-19433-7_40 Accessed 2025-11-24
- [28] Alakuijala, J., Farruggia, A., Ferragina, P., Kliuchnikov, E., Obryk, R., Szabadka, Z., Vandevenne, L.: Brotli: A General-Purpose Data Compressor. *ACM Transactions on Information Systems* **37**(1), 1–30 (2019) <https://doi.org/10.1145/3231935>

- [29] Fernández, J.D., Martínez-Prieto, M.A., Gutiérrez, C., Polleres, A., Arias, M.: Binary rdf representation for publication and exchange (hdt). *Web Semantics: Science, Services and Agents on the World Wide Web* **19**, 22–41 (2013)
- [30] Arenas-Guerrero, J., Ferrada, S.: COTTAS: Columnar triple table storage for efficient and compressed RDF management. In: *The Semantic Web – ISWC 2025. Lecture Notes in Computer Science*, vol. 16141, pp. 313–331. Springer, Cham (2026). https://doi.org/10.1007/978-3-032-09530-5_18 . https://doi.org/10.1007/978-3-032-09530-5_18
- [31] Menotti, L., Cazzaro, M., Rueda, M., Gut, I.G., Silvello, G.: HERO-Genomics: Bridging Genomic Data and Ontological Modelling. *CEUR Workshop Proceedings*, Barcelona, Spain (2025). https://www.dei.unipd.it/~silvello/papers/2025.MCS_SWAT4HCLS.pdf
- [32] Wagner, A.H., Hart, R., Babb, L., Kuzma, K., Schuilenburg, H., Stevenson, J., O'Neill, T., Epic, M.-T., Holmes, B., Moyer, E., Baudis, M., Robinson, P., Cline, M., Patel, R.Y., Freimuth, R.R.: ga4gh/vrs 2.0.1. Zenodo (2025). <https://doi.org/10.5281/zenodo.15059674> . <https://doi.org/10.5281/zenodo.15059674> Accessed 2026-03-02
- [33] Diefenbach, D., Giménez-García, J.M.: Hdtcat: Let's make hdt generation scale. In: *The Semantic Web – ISWC 2020. Lecture Notes in Computer Science*, vol. 12507, pp. 18–33. Springer, ??? (2020). https://doi.org/10.1007/978-3-030-62466-8_2
- [34] Arenas-Guerrero, J.: pycottas: Python COTTAS library for compressing and querying RDF. RDFLib store backend, triple-pattern interface, and COTTAS conversion utilities (2026). <https://pypi.org/project/pycottas/> Accessed 2026-08-31
- [35] Personal Genome Project (2005). https://my.pgp-hms.org/public_genetic_data Accessed 2025-11-24
- [36] NIST Genome in a Bottle (2025). <https://www.nist.gov/programs-projects/genome-bottle> Accessed 2025-11-24
- [37] The 1000 Genomes Project Consortium: A global reference for human genetic variation. *Nature* **526**(7571), 68–74 (2015) <https://doi.org/10.1038/nature15393>
- [38] Ebert, P., Audano, P.A., Zhu, Q., Rodriguez-Martin, B., Porubsky, D., Bonder, M.J., Sulovari, A., Ebler, J., Zhou, W., Serra Mari, R., *et al.*: Haplotype-resolved diverse human genomes and integrated analysis of structural variation. *Science* **372**(6537), 7117 (2021) <https://doi.org/10.1126/science.abf7117>
- [39] Wagner, J., Olson, N.D., Harris, L., Khan, Z., Farek, J., Mahmoud, M., Stankovic, A., Kovacevic, V., Yoo, B., Miller, N., *et al.*: Benchmarking challenging small

- variants with linked and long reads. *Cell Genomics* **2**(5), 100128 (2022) <https://doi.org/10.1016/j.xgen.2022.100128>
- [40] National Center for Biotechnology Information: dbSNP Build 156 VCF Release (GRCh38). Header declares `##fileformat=VCFv4.2`; accessed 8 September 2026 (2026). https://ftp.ncbi.nlm.nih.gov/snp/latest_release/VCF/
 - [41] Genome Aggregation Database Consortium: gnomAD v4.1 Genome Aggregation Database VCF Release. Header declares `##fileformat=VCFv4.2`; accessed 8 September 2026 (2026). <https://gnomad.broadinstitute.org/downloads>
 - [42] National Center for Biotechnology Information: ClinVar VCF Release (GRCh38). Header declares `##fileformat=VCFv4.1`; accessed 8 September 2026 (2026). https://ftp.ncbi.nlm.nih.gov/pub/clinvar/vcf_GRCh38/
 - [43] International Genome Sample Resource: 1000 Genomes 30x GRCh38 Phased Call Set (3,202 Samples). Header declares `##fileformat=VCFv4.1`; accessed 8 September 2026 (2026). https://ftp.1000genomes.ebi.ac.uk/vol1/ftp/data_collections/1000G_2504_high_coverage/working/20201028_3202_phased/
 - [44] Ensembl: Ensembl Variation VCF Release. Header declares `##fileformat=VCFv4.1`; accessed 8 September 2026 (2026). https://ftp.ensembl.org/pub/current_variation/vcf/homo_sapiens/
 - [45] Danecek, P., Auton, A., Abecasis, G., Albers, C.A., Banks, E., DePristo, M.A., Handsaker, R.E., Lunter, G., Marth, G.T., Sherry, S.T., McVean, G., Durbin, R., 1000 Genomes Project Analysis Group: The variant call format and VCFtools. *Bioinformatics* **27**(15), 2156–2158 (2011) <https://doi.org/10.1093/bioinformatics/btr330>
 - [46] Pedersen, B.S., Quinlan, A.R.: cyvcf2: fast, flexible variant analysis with Python. *Bioinformatics* **33**(12), 1867–1869 (2017) <https://doi.org/10.1093/bioinformatics/btx057>
 - [47] Taelman, R., Herwegen, J.V., Sande, M.V., Verborgh, R.: Comunica: A modular SPARQL query engine for the web. In: *Proceedings of the 17th International Semantic Web Conference*, pp. 239–255. Springer, ??? (2018). https://doi.org/10.1007/978-3-030-00668-6_15 . <https://comunica.dev/>
 - [48] Comunica Association: Querying local files from the command line (2026). https://comunica.dev/docs/query/getting_started/query_cli_file/ Accessed 2026-08-31
 - [49] Comunica Association: HDT: Querying HDT files with Comunica (2026). <https://comunica.dev/docs/query/advanced/hdt/> Accessed 2026-08-31
 - [50] Ashburner, M., Ball, C.A., Blake, J.A., Botstein, D., Butler, H., Cherry, J.M.,

- Davis, A.P., Dolinski, K., Dwight, S.S., Eppig, J.T., Harris, M.A., Hill, D.P., Issel-Tarver, L., Kasarskis, A., Lewis, S., Matese, J.C., Richardson, J.E., Ringwald, M., Rubin, G.M., Sherlock, G.: Gene Ontology: tool for the unification of biology. *Nature Genetics* **25**(1), 25–29 (2000) <https://doi.org/10.1038/75556>
- [51] Sowiński, P., Bogacka, K., Danilenka, A., Kozlov, N.: Jelly: a fast and convenient rdf serialization format. In: Joint Proceedings of Posters, Demos, Workshops, and Tutorials of the 21st International Conference on Semantic Systems Co-located with 21st International Conference on Semantic Systems (SEMANTiCS 2025), Vienna, Austria, September 3-5, 2025. *CEUR Workshop Proceedings*, vol. 4064. CEUR-WS.org, ??? (2025). <https://ceur-ws.org/Vol-4064/SEMDEV-paper3.pdf>
- [52] Jiménez, R.C., Kuzak, M., Alhamdoosh, M., Barker, M., Batut, B., Borg, M., Capella-Gutiérrez, S., Chue Hong, N.P., Cook, M., Corpas, M., Flannery, M., Garcia, L., Gelpí, J.L., Gladman, S., Goble, C., González Ferreiro, M., Gonzalez-Beltran, A., Griffin, P.C., Grüning, B., Hagberg, J., Holub, P., Hooft, R., Ison, J., Jones, S., Kaiser, F., Kim, S., Krafczyk, M., Krystalli, A., Lapp, H., Lonie, A., Marek, C., Mazanetz, M.P., Nenadic, A., Petersen, W., Plomp, E., Pomazkov, M., Ram, K., Soiland-Reyes, S., Sokolov, A., Struck, A., Suchacki, R., Vos, R., Williams, A.R., Brack, P., Muelas, M.W., Diehl, A.D., Poline, J.-B., Atwood, T.K., Mansurov, A., Soldatova, L.N., Kingsford, C., Kersey, P.J., McDowall, M.D., Østeras, M., Guijarro-Clarke, C., Crouch, S.: Four simple recommendations to encourage best practices in research software. *F1000Research* **6**, 876 (2017) <https://doi.org/10.12688/f1000research.11407.1>